

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



PROGRAMMING INTEGRATION PROJECT

TOPIC: DEVELOP AN APPLICATION TO SUPPORT
DATA ANALYSIS IN BANKING

Banking Data Analysis Application Development

Instructor(s): Assoc. Prof. Võ Thị Ngọc Châu

Student(s): Nguyễn Ngọc Cát Hà ID 2352288

Phạm Viết Hào ID 2352297

Trịnh Gia Hiệp ID 2352343

Nguyễn Thanh Hiếu ID 2352331

HO CHI MINH CITY, NOVEMBER 2025



Member list & Workload

No.	Full name	Student ID	Contribution	Tasks
1	Nguyễn Ngọc Cát Hà	2352288	100%	Customer Churn Analytics
2	Phạm Viết Hào	2352297	100%	Credit Risk Assessment
3	Trịnh Gia Hiệp	2352343	100%	Real-time Fraud Detection
4	Nguyễn Thanh Hiếu	2352331	100%	Data Quality Dashboard Historical Trend Analysis



Contents

1	Project Introduction	9
1.1	Application Area Research	9
1.1.1	General	9
1.1.2	The benefits of big data in banking	9
1.1.3	The future of big data in banking	10
1.2	Problem Identification - Banking Data Analysis Problem	10
1.2.1	Core Problem Areas	11
1.2.2	Problem Statement	12
1.3	User & Stakeholder Analysis	12
1.3.1	Application Overview	12
1.3.2	Core User Groups	13
1.3.3	Success Metrics by Stakeholder Groups	14
1.4	Technologies and Tools	14
1.4.1	Front-end	14
1.4.2	Back-end	15
1.4.3	Processing Data	15
1.4.4	Database	15
1.4.5	Miscellaneous	15
2	Bussiness Process Model	16
2.1	Data Sources	16
2.2	Process Flow by User Group	17
2.2.1	Data Analysts	17
2.2.2	Risk & Credit Officers	17
2.2.3	Fraud Operations	17
2.2.4	Compliance Officers	17
2.2.5	IT Operations	17
2.3	Process Interactions	18
3	Features	18
3.1	Credit Risk & Scoring	18
3.1.1	Feature Description	18
3.1.2	Functional Requirements	18
3.1.3	Non-functional Requirements	19
3.1.4	Data Requirements	20



3.1.5	Requirements Analysis	23
3.1.6	Project Type Mapping	24
3.2	Customer Churn Analytics Platform	24
3.2.1	Feature Description	24
3.2.2	Functional Requirements	24
3.2.3	Non-functional Requirements	25
3.2.4	Data Requirements	26
3.2.5	Requirements Analysis	28
3.2.6	Project Type Mapping	29
3.3	Real-time Fraud Detection & Prevention System	29
3.3.1	Feature Description	29
3.3.2	Functional Requirements	29
3.3.3	Non-functional Requirements	30
3.3.4	Data Requirements	31
3.3.5	Requirements Analysis	32
3.3.6	Project Type Mapping	33
3.4	Data Quality Dashboard	34
3.4.1	Feature Description	34
3.4.2	Functional Requirements	34
3.4.3	Non-functional Requirements	34
3.4.4	Data Requirements	35
3.4.5	Requirements Analysis	38
3.4.6	Project Type Mapping	40
3.5	Historical Trend Analysis	40
3.5.1	Feature Description	40
3.5.2	Functional Requirements	41
3.5.3	Non-functional Requirements	42
3.5.4	Data Requirements	42
3.5.5	Requirements Analysis	45
3.5.6	Project Type Mapping	47
4	Technology Stack and Comparison	47
4.1	Machine Learning Models	47
4.2	Databases	48
4.3	Frontend Frameworks	49
4.4	Backend API Frameworks	51



4.5	Real-time Communication	52
4.6	Message Queue Systems	53
4.7	Notification Systems	55
4.8	Explainability Tools	56
4.9	Time-Series Processing	57
4.10	Technology Stack Summary	58
5	Use Case Diagram	61
6	System Architecture	62
6.1	Layered Architecture Summary	62
6.1.1	Presentation Layer	62
6.1.2	Application Layer (Microservices)	63
6.1.3	Machine Learning Layer	63
6.1.4	Integration Layer	63
6.1.5	Data Layer	63
6.1.6	Infrastructure Layer	64
6.2	Visualization	64
6.3	External Systems	64
6.4	Quality Attributes	65
7	Database Design	66
7.1	Entity-Relationship Model	66
7.2	Relational Schema	67
8	Data Architecture Comparison	68
9	UI/UX	70
10	Implementation	71
10.1	Fraud Detection Implementation	71
10.1.1	Data Preparation	71
10.1.2	Model Training	73
10.1.3	Model Evaluation	73
10.2	Credit Score Classification Implementation	74
10.2.1	Data Preparation	74
10.2.2	Model Training	77
10.2.3	Model Persistence	78



10.2.4	Model Evaluation	79
10.2.5	Model Artifacts	79
10.2.6	Source Code	79
10.3	Customer Churn Prediction System	80
10.3.1	Data Preparation	80
10.3.2	Feature Engineering	81
10.3.3	Model Development	81
10.3.4	Model Evaluation	82
10.3.5	Prediction and Submission Generation	82

List of Figures

1.1	Big Data Uses Cases in Banking ^[24]	10
2.1	Business Process Model	16
5.1	Use-case Diagram	61
6.1	Visualization of system architecture	64
7.1	Entity-Relationship Diagram	66
7.2	Relational Schema Mapping	67
9.1	Visualization of User Login Screen	70
9.2	Visualization of User Main Screen	71
9.3	Visualization of User Main Screen in Dark mode	71

List of Tables

1.1	Core user groups	13
1.2	Stakeholders Success Metrics	14
3.1	Functional Requirements for Credit Risk & Scoring	18
3.2	Non-functional Requirements for Credit Risk & Scoring	19
3.3	Data Requirements for Credit Risk & Scoring	20
3.4	Project Type Mapping for Credit Risk Assessment & Scoring	24
3.5	Functional Requirements for Customer Churn Analytics	24
3.6	Non-functional Requirements for Customer Churn Analytics	25
3.7	Data Requirements for Customer Churn Analytics	26
3.8	Project Type Mapping of Customer Churn Analytics	29
3.9	Functional Requirements for Real-time Fraud Detection & Prevention System	30



3.10 Non-functional Requirements for Real-time Fraud Detection & Prevention System	30
3.11 Data Requirements for Real-time Fraud Detection & Prevention System . .	31
3.12 Project Type Mapping of Real-time Fraud Detection & Prevention System .	33
3.13 Functional Requirements for Data Quality Dashboard	34
3.14 Non-functional Requirements for Data Quality Dashboard	35
3.15 Data Requirements for Data Quality Dashboard	35
3.16 Project Type Mapping of Data Quality Dashboard	40
3.17 Functional Requirements for Historical Trend Analysis	41
3.18 Non-functional Requirements for Historical Trend Analysis	42
3.19 Data Requirements for Historical Trend Analysis	43
3.20 Project Type Mapping of Historical Trend Analysis	47
4.1 Machine Learning Model Comparison	48
4.2 Database Comparison	49
4.10 Final Technology Stack	59
8.1 Data Architecture Comparison	68

Listings

10.1 Load Dataset	72
10.2 Balance Dataset with Undersampling	72
10.3 Split and Scale Data	73
10.4 Train Logistic Regression Model	73
10.5 Evaluate Model Performance	73
10.6 Load and Initialize Dataset	74
10.7 Feature Group Definitions	75
10.8 Clean and Handle Outliers	75
10.9 Median Imputation	76
10.10 Encode Categorical Variables	76
10.11 Feature-Target Separation and Split	77
10.12 Train XGBoost Classifier	77
10.13 Save Model and Artifacts	78
10.14 Evaluate Model Performance	79
10.15 Load Datasets	80
10.16 Target Distribution Visualization	80



10.17Encoding and Data Split	81
10.18Tuned XGBoost Classifier	81
10.19Confusion Matrix	82
10.20Generate Submission File	82



1 Project Introduction

1.1 Application Area Research

1.1.1 General

The banking and finance industry today is heavily influenced by Big Data and digital technology. Banks have to manage huge amounts of data from customer transactions, credit records, card services, Internet banking, and external data sources.

According to McKinsey's Global Banking Annual Review (2024), only 14% of banks currently achieve performance levels that create sustainable value, highlighting the critical need for data-driven transformation in the banking sector. The report emphasizes that banks spend the highest proportion of revenues on technology across all sectors, yet many struggle to realize expected returns from these investments. Research shows that banks spend 6-12% of their annual technology budget specifically on data initiatives, reflecting the industry's recognition of data as a competitive differentiator^[20].

1.1.2 The benefits of big data in banking

Big data gives you a full view on your business: from customer behavior patterns to internal process efficiency and even broader market trends. This means you can make informed, data-driven decisions and, subsequently, obtain business results.

It allows you to optimize and streamline your internal processes with the help of machine learning and AI. As a result, you get a significant performance boost and reduced operating costs.

Big data in banking and financial services is pivotal to improving the level of client satisfaction, as data has always been the primary resource for developing and offering personalized solutions to each client.

Customer segmentation in banking using big data may be helpful for classifying clients based, for instance, on their financial activities. This can be a foundation for analyzing client behavior, launching more effective marketing campaigns, and finding the most efficient approach to different groups.

Big data in banking can also be used for developing algorithmic trading systems that can make more efficient trading decisions faster and more consistently than human traders.

Big data analytics in banking can be used to enhance your cybersecurity and reduce risks. By using intelligent algorithms, you can detect fraud and prevent potentially malicious actions.

Big Data Use Cases in Banking



Figure 1.1: Big Data Uses Cases in Banking^[24]

1.1.3 The future of big data in banking

Big data becomes increasingly crucial in any area of the banking industry. Implementing big data in banking and finance is arguably the only way to regain control over the client flow while maintaining an excellent level of service delivery, which was showcased in multiple aspects.

Banks need to rethink their operations and adopt data-driven approaches if they want to stay relevant and competitive. Plus, big data in the banking sector can help you improve and grow your business^[?].

1.2 Problem Identification - Banking Data Analysis Problem

The banking industry faces major challenges in using data properly for better operations, customer satisfaction, and following regulations. Modern banks operate many different systems with data stored in different formats and places, creating big problems for complete data analysis and decision-making.

1.2.1 Core Problem Areas

Data Integration and Quality Management Major banks like Wells Fargo and Citigroup have faced regulatory fines due to data management problems and systems that cannot handle large amounts of data^[12]. Banks struggle with many data sources that make integration difficult, making it hard to decide which sources to use first when combining different types of data^[26].

Data quality management involves checking that bank data is accurate, complete, and consistent across all systems. Banks face problems with missing information, duplicate records, and inconsistent data formats that lead to wrong decisions and regulatory violations^[29]. For example, a customer's information might show different account balances in different systems, or transaction records might be incomplete or contain errors. According to Deloitte's Banking & Capital Markets Data Analytics Survey (2024), US banks planned to invest over \$5 billion in data initiatives in 2023, with 94% of banking leaders identifying data accuracy and reliability as their top priority. However, only 52% of banks have successfully migrated more than half of their data to cloud platforms, indicating significant integration challenges remain^[8].

Legacy System Integration and Modernization One of the biggest challenges for banks is connecting old systems with new digital solutions, as data stuck in old systems prevents integration with modern APIs^[23]. Old systems lack the infrastructure to handle modern banking analytics and big data processing^[31].

Legacy systems are older computer systems that banks have used for many years to handle core banking functions like account management, transaction processing, and customer records. These systems often use outdated technology and cannot easily connect to modern applications or share data with newer systems. Banks need to make these old systems communicate with newer systems to provide access to all necessary data for third-party applications, but there are no universal technical standards for how these integrations should work^[30]. This creates problems when banks want to offer mobile banking, online services, or comply with Open Banking regulations that require sharing customer data with authorized third parties.

Customer Behavior Analysis and Personalization Banks need better ways to analyze customer behavior to improve operations and build customer loyalty, with machine learning and natural language processing being widely used for better customer experiences^[28]. Banks need a complete view of customers by combining data from multiple

sources to stay competitive^[17].

Customer behavior analysis means understanding how customers use banking services, what products they prefer, when they make transactions, and what financial needs they have. Banks need to analyze customer spending habits, loan repayment patterns, savings trends, and transaction frequency to understand individual customer preferences and provide personalized services^[27]. For example, banks want to know if a customer typically saves money at the end of each month, prefers mobile banking over branch visits, or might be interested in a loan based on their spending patterns. Banks face customer attrition rates of up to 15% annually and need to anticipate customer needs to provide timely, relevant services that prevent customers from switching banks^[14].

Risk Assessment and Fraud Detection Traditional fraud detection systems create too many false alarms and cannot adapt to new fraud patterns, while AI systems can monitor huge amounts of transactions automatically and provide real-time analysis^[16]. Banks need new tools and techniques for predicting risks, with automated machine learning helping find the best models for specific situations^[6].

1.2.2 Problem Statement

Banks need an integrated data analysis platform that solves the main problems of combining data, checking data quality, analyzing customer behavior, and managing risk. The proposed banking data analysis application will provide complete solutions for bringing together data from multiple sources, automatic quality monitoring, customer grouping and behavior prediction, and smart risk assessment tools.

We hope to build a platform that will help banks turn scattered data into useful insights, improve operations through automatic processes, enhance customer experience through personalized services, and strengthen risk management through prediction tools and real-time monitoring.

1.3 User & Stakeholder Analysis

1.3.1 Application Overview

A banking data-analytics platform typically ingests large volumes of internal (transactions, CRM, risk) and external (open-banking, bureau, social) data, then transforms it into dashboards, models and APIs that support:

- Customer segmentation & personalization



- Fraud detection & AML (anti money laundering) monitoring
- Credit-risk scoring & portfolio optimisation
- Real-time risk, liquidity and ALM (asset liability management) reporting
- Marketing, cross-sell and churn prediction
- Regulatory compliance, stress-testing and ESG metrics

Each capability attracts its own set of users and stakeholders.

1.3.2 Core User Groups

These are the main users of data in banking:

Table 1.1: Core user groups

User Group	Typical Roles & Goals	Key Tasks	Critical Needs
Bank Analysts	Analyze financial data and customer trends	Divide and analyze customer groups, evaluate how groups react to marketing	Highly detailed customer attributes, self-service Business Intelligence
Risk & Credit Modellers	Staff of CRO	Build default models, monitoring	Historical data documentation, model governance
Fraud operations	Fraud detections, AML (anti money laundering)	Real-time alert, case management	Low update latency, explainability
Relationship managers	Providing financial advice, managing clients accounts	360° customer view, product prompts	Simplified UI, mobile access
Marketing teams	Product Owners	Track campaign ROI, propensity models	Unified metrics, export to Marketing Tech



User Group	Typical Roles & Goals	Key Tasks	Critical Needs
Data scientist	Bank lab, calculate Cost of Equity	Experiment with new data sources	Sandbox, notebook integration
Compliance officers	Regulatory, audit	Reports	Traceable calculations, audit trails
Executives	Directive Board	KPI dashboards, risk appetite	High-level visualisation
External partners	Financial Tech companies, ...	API consumption	Secure, well-documented APIs

1.3.3 Success Metrics by Stakeholder Groups

Table 1.2: Stakeholders Success Metrics

Stakeholder Group	Success Metric
Fraud Ops	Fraud loss rate
Risk Modeller	Model deployment time
Marketing	Cross-sell conversion
IT Ops	Platform uptime
Compliance	Regulatory breaches
Executives	Cost-to-income ratio

1.4 Technologies and Tools

1.4.1 Front-end

Tools: HTML, CSS, Javascript, React

Advantages:

- React modularity is perfect for dashboard components such as charts, tables, etc.
- React has great compatibility with multiple platforms^[32].
- All are universal, widely-use and flexible to work with.



1.4.2 Back-end

Tools: Node.js, FastAPI

Advantages:

- Node.js is low cost, easy to scale, and has a rich package ecosystem too work with.
- FastAPI is fast, modern, easy to set up and intuitive^[10].

1.4.3 Processing Data

Tools: Python, Kaggle

Advantages:

- Python has a large amount of libraries for mathematical operations and data visualization.
- Kaggle offers a strong cloud service to train your own AI model on their platform for free.

1.4.4 Database

Tools: PostgreSQL

Advantages:

- PostgreSQL is a low cost, open source database that focuses on reliability, security, extensibility, etc.
- PostgreSQL also supports multiple file formats such as JSON and XML^[25].

1.4.5 Miscellaneous

Tools: Git + Github, Kaggle dataset

Advantages:

- Git and Github are industry standard in version control, and provide excellent results.
- Kaggle provides quality datasets for testing and a great source of data for prototypes.

2 Bussiness Process Model

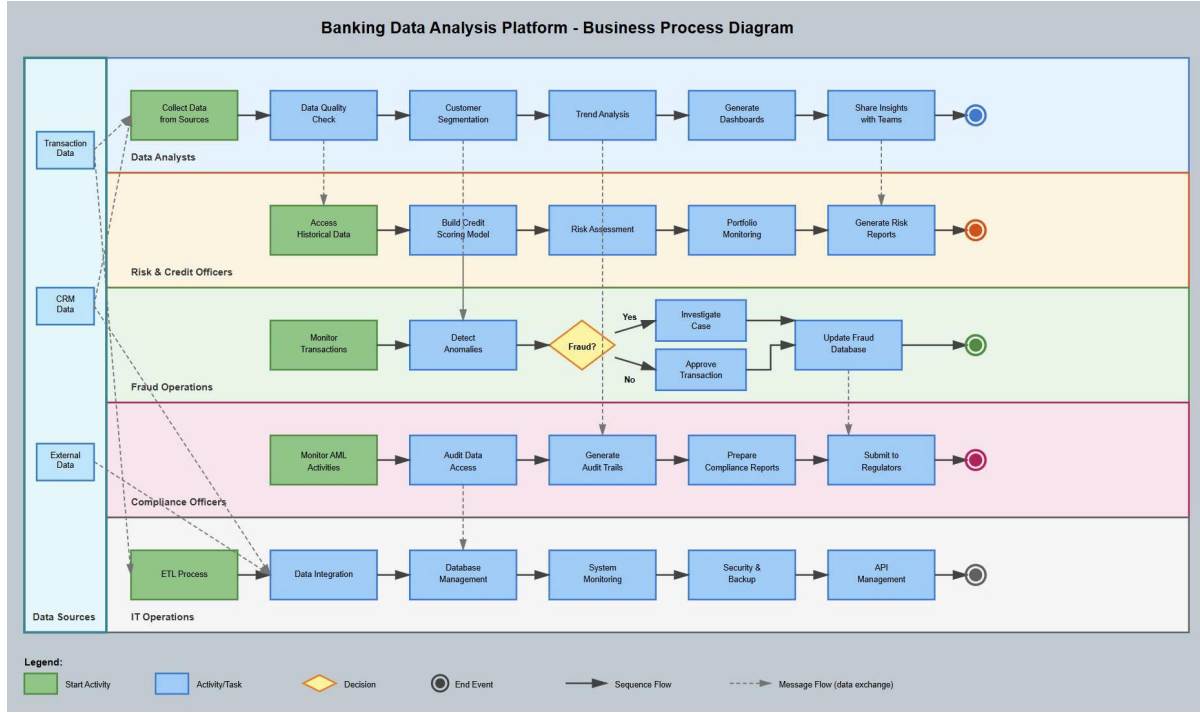


Figure 2.1: Business Process Model

The business process model for the Banking Data Analysis Platform illustrates how data flows through different organizational units to deliver analytical insights. The model uses Business Process Model and Notation (BPMN) 2.0 standards with horizontal swimlanes to clearly delineate responsibilities across five primary user groups.

2.1 Data Sources

The process begins with data ingestion from three primary sources:

- **Transaction Data:** Customer payments, transfers, deposits, and withdrawals from core banking systems
- **CRM Data:** Customer profiles, service usage patterns, and interaction histories
- **External Data:** Open banking APIs, credit bureau reports, social media data, and third-party market information

These diverse data sources are consolidated and processed through the platform to support various analytical activities across different departments.



2.2 Process Flow by User Group

2.2.1 Data Analysts

Data Analysts initiate the analytical pipeline by collecting data from multiple sources, performing quality checks to ensure accuracy and completeness. They then segment customers based on behavior and demographics, conduct trend analysis, generate dashboards, and share insights with other teams. This lane serves as the foundation for business intelligence across the organization.

2.2.2 Risk & Credit Officers

Risk and Credit Officers access historical data to build credit scoring models that evaluate customer creditworthiness. They continuously monitor portfolio risk, assess default probabilities, and generate risk reports for management decision-making. Their processes rely heavily on validated data from the Data Analysts lane.

2.2.3 Fraud Operations

The Fraud Operations team monitors transactions in real-time, detecting anomalies that may indicate fraudulent activity. When suspicious patterns are identified, a decision point determines whether to investigate the case or approve the transaction. All fraud cases are logged in a centralized database for pattern analysis and regulatory reporting.

2.2.4 Compliance Officers

Compliance Officers monitor Anti-Money Laundering (AML) activities, audit data access patterns, and generate comprehensive audit trails. They prepare compliance reports and submit them to regulatory bodies, ensuring the bank meets all legal and regulatory requirements. Their work is critical for maintaining the bank's license to operate.

2.2.5 IT Operations

IT Operations provides the technical foundation by managing ETL (Extract, Transform, Load) processes, ensuring data integration across systems, maintaining database performance, monitoring system health, implementing security measures, and managing APIs for internal and external integrations.

2.3 Process Interactions

The business process model demonstrates several key interactions between lanes:

- Data flows from IT Operations to all other user groups through message flows (indicated by dashed lines)
- Data Analysts share quality-checked data with Risk Officers and Fraud Operations
- Fraud Operations updates are communicated to Compliance Officers for regulatory reporting
- All processes ultimately rely on the infrastructure maintained by IT Operations

This collaborative model ensures that data-driven insights are generated efficiently while maintaining data quality, security, and regulatory compliance throughout the analytical lifecycle.

3 Features

3.1 Credit Risk & Scoring

3.1.1 Feature Description

The Credit Risk Assessment and Scoring System is designed to evaluate the financial reliability and creditworthiness of individuals or organizations applying for loans or credit facilities. It automates the process of data collection, risk analysis, scoring, and grading, helping financial institutions make informed and consistent lending decisions. The system combines quantitative financial data (such as income, assets, debt ratio) and qualitative factors (such as credit history, industry stability, and management quality) to compute a credit score. Based on predefined or configurable thresholds, the system then assigns a risk grade — for example, Low, Medium, or High risk — and produces a comprehensive credit report for analysts or loan officers to review.

3.1.2 Functional Requirements

Table 3.1: Functional Requirements for Credit Risk & Scoring

ID	Requirement	Description	Reasoning
----	-------------	-------------	-----------



CRAS-2.1	Risk Scoring	Calculate credit risk scores with probability of default predictions ranging 0-1.	Ensures accurate, data-driven risk evaluation using robust predictive models.
CRAS-2.2	Credit Classification	Classify borrowers into risk categories (Low, Medium, High, Very High) based on calculated scores and thresholds.	Provides clear and interpretable categories for decision-making.
CRAS-2.3	Stand Alone Inputs	The system must allow users to manually input data for assessment.	Enables use even without automated data sources.
CRAS-2.4	Decision Support	Recommend approve/reject/manual review actions based on risk thresholds and business rules.	Improves decision consistency and reduces bias.
CRAS-2.5	Report Generation	Generate comprehensive risk assessment reports including borrower profile, risk factors, score breakdown, and recommendations.	Ensures transparency and supports audits or reviews.
CRAS-2.6	API Integration	Expose RESTful APIs for integration with loan origination systems, mobile apps, and third-party platforms.	Enables scalability and easy system integration.

3.1.3 Non-functional Requirements

Table 3.2: Non-functional Requirements for Credit Risk & Scoring

ID	Requirement	Description	Reasoning
----	-------------	-------------	-----------

CRAS-3.1	Performance	The system must process a complete credit risk assessment (including data retrieval and scoring) for a standard credit application.	Ensures fast user response and supports real-time decision-making.
CRAS-3.2	Usability	The user interface must be intuitive and easy to navigate, allowing new users to complete basic operations with less than 2 hours of training.	Improves user adoption and reduces training effort.
CRAS-3.3	Maintainability	The system's architecture should be modular to allow easy updates to scoring models, thresholds, and user interfaces without major downtime.	Simplifies updates and lowers maintenance costs.
CRAS-3.4	Compatibility	The system should operate across major web browsers (Chrome, Firefox, Edge) and integrate smoothly with existing banking systems and APIs.	Ensures system accessibility and smooth integration.

3.1.4 Data Requirements

Table 3.3: Data Requirements for Credit Risk & Scoring

Requirement	Description	Reasoning
-------------	-------------	-----------



Requirement	Description	Reasoning
Data Source	Credit Score Classification Dataset (Kaggle) Link: Credit Score Classification Dataset • Size: 100,000 records • Score Distribution: Good (17.8%), Standard (53.2%), Poor (29.0%) • Fields: ID, Customer_ID, Month, Name, Age, SSN, Occupation, Annual_Income, Monthly_Inhand_Salary, Num_Bank_Accounts, Num_Credit_Card, Interest_Rate • Format: CSV (train.csv, test.csv) • Data Quality: Contains missing values, mixed data types, requires extensive preprocessing	Kaggle dataset provides comprehensive labeled data spanning demographics, financial behavior, payment history, and credit utilization patterns for multi-class classification model training. Real-world banking systems can integrate this model for ongoing credit assessment, loan approval automation, and risk-based pricing decisions.



Requirement	Description	Reasoning
Input Data	• Demographics: Age, Occupation, Annual Income • Account Info: Num_Bank_Accounts, Num_Credit_Card, Credit_Mix, Credit_Utilization_Ratio, Outstanding_Debt • Behavioral Data: Delay_from_due_date, Num_of_Delayed_Payment, Changed_Credit_Limit, Payment_of_Min_Amount • Financial Ratios: Monthly_Inhand_Salary, Total_EMI_per_month, Monthly_Balance	Multi-dimensional attributes enable comprehensive borrower profiling and support predictive analytics for credit-worthiness.
Output Data	• Predictions: Credit Score class (<i>Good / Standard / Poor</i>) with confidence level • Explanations: Top contributing factors to score (e.g., debt ratio, late payments) • Visualizations: Distribution of scores, feature importance, risk trend charts • Reports: Individual and group-level risk summaries	Rich outputs assist in credit decisioning, customer segmentation, and portfolio risk management.

3.1.5 Requirements Analysis

Doability: HIGHLY DOABLE

The development of a Credit Risk Assessment and Scoring System using the Kaggle *Credit Score Classification Dataset* is highly feasible. The dataset provides over 100,000 labeled borrower records with key financial and behavioral attributes suitable for model training and validation.

- **Datasets:** Publicly available datasets such as the Kaggle Credit Score Classification Dataset contain diverse borrower information (income, debt ratio, delayed payments, credit utilization) supporting supervised learning and performance benchmarking.
- **Algorithms:** Established machine learning models — Logistic Regression, Random Forest, XGBoost, and Decision Trees — are proven effective for predicting default likelihood and credit score classification.
- **Libraries & Tools:** The system can be efficiently developed using Python's `pandas`, `NumPy`, `scikit-learn`, and `matplotlib` for model training and visualization. For deployment, web frameworks like `Flask` or `Django` with a front-end built in `React.js` enable interactive user interfaces and explainable AI visualizations.

Testability: HIGHLY TESTABLE

The system's accuracy, robustness, and explainability can be objectively validated using quantitative metrics and structured testing procedures.

- **Evaluation Metrics:** Precision, Recall, F1-Score, Accuracy, and AUC-ROC are applied to assess classification performance and balance between Type I and Type II errors.
- **Validation Methods:** K-Fold Cross-Validation ensures generalization across borrower groups, while confusion matrices visualize the model's classification reliability across the credit score categories (*Good*, *Standard*, *Poor*).
- **Test Coverage:** Model testing with over 100 borrower profiles spanning low-risk, medium-risk, and high-risk segments verifies stable performance and fairness across different credit groups.

Importance: CRITICAL (9/10)

Credit risk assessment is a fundamental function within financial institutions and fintech systems. It directly influences loan approval decisions, interest rate pricing, and portfolio risk exposure.



- It supports regulatory compliance under the Basel II/III frameworks^[3] by quantifying borrower default probabilities.
- It underpins automated credit scoring, real-time lending, and risk-based pricing strategies in digital banking ecosystems.
- It provides the analytical backbone for sustainable credit growth and operational risk reduction in financial institutions.

3.1.6 Project Type Mapping

Table 3.4: Project Type Mapping for Credit Risk Assessment & Scoring

Category	Description
DE (Data Engineering)	ETL pipelines to aggregate credit data from multiple sources; supports real-time streaming for model updates.
IS (Information Systems)	Credit Scoring Dashboard for loan officers to view and act on model outputs.
AI (Artificial Intelligence)	Machine learning models for default prediction, feature selection, and continuous re-training.

3.2 Customer Churn Analytics Platform

3.2.1 Feature Description

Comprehensive machine learning system that predicts customer churn probability, classifies risk levels, and provides real-time dashboard with alerts for proactive retention strategies. The system combines predictive analytics with executive visualization to identify at-risk customers and enable timely intervention.

3.2.2 Functional Requirements

Table 3.5: Functional Requirements for Customer Churn Analytics

ID	Requirement	Description	Reasoning
----	-------------	-------------	-----------



CCAP-2.1	Churn Prediction & Risk Classification	ML model calculates churn probability (0-100%) and classifies into Low (0-30%), Medium (31-70%), High (71-100%) risk tiers.	Detailed probability enables customer prioritization; simple classification helps teams take action.
CCAP-2.2	Real-time Dashboard & Alerts	Dashboard displays churn rates, trends, segments; automatically sends alerts (email, SMS, in-app) when VIP customers exceed 70% churn probability.	Instant visibility enables leadership decisions; quick alerts minimize revenue loss.
CCAP-2.3	Prediction Explanation	Provides top 3-5 factors influencing predictions with feature importance scores.	Explainability helps understand causes and design retention strategies.
CCAP-2.4	Batch & Real-time Processing	Batch: processes 1M+ customers daily; Real-time API: instant predictions (< 2 sec).	Regular updates for campaigns; real-time for customer interactions.
CCAP-2.5	Interactive Analytics	Trend analysis, segment drill-down (age, geography, product), filters, PDF/Excel export.	Self-service analytics enables deep analysis without technical support.

3.2.3 Non-functional Requirements

Table 3.6: Non-functional Requirements for Customer Churn Analytics

ID	Requirement	Description	Reasoning
CCAP-3.1	Performance	Dashboard should load without noticeable delay under normal conditions; API predictions should respond in near real-time	Ensures smooth user experience and operational efficiency



CCAP-3.2	Accuracy	Model accuracy $\geq 70\%$; False Positive Rate $< 15\%$	Actionable insights; minimizes wasted retention spending
CCAP-3.3	Availability	High availability with minimal planned down-time; Support for typical enterprise user loads	Ensures access when needed; supports distributed teams
CCAP-3.4	Scalability	System should handle current customer base with room for growth; horizontal scaling; model monitoring with drift detection	Business growth without redesign; sustained model quality

3.2.4 Data Requirements

Table 3.7: Data Requirements for Customer Churn Analytics

Requirement	Description	Reasoning
Data Source	Bank Customer Churn Dataset (Kaggle) Link: Bank Customer Churn Dataset • Size: 10,000+ customer records • Churn Rate: $\sim 20\%$ (realistic banking churn rate) • Fields: CustomerID, Age, Gender, Geography, CreditScore, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember, EstimatedSalary, Exited • Format: CSV	Kaggle dataset offers labeled data for churn model training, while real banking systems provide production features for ongoing prediction and behavior analysis.



Requirement	Description	Reasoning
Input Data	• Demographics: Age, Gender, Geography, Estimated Salary, Tenure • Account Info: CreditScore, Balance, NumOfProducts, HasCrCard, IsActiveMember • Transaction Behavior: Monthly volume, average amount, spending categories (3+ years) • Digital Engagement: Login frequency, session duration, app usage, last login • Financial Health: Credit changes, overdraft frequency, payment delays, balance trends • Temporal Features: Account age, days since last transaction, seasonal patterns	Multi-dimensional data enables comprehensive analysis and intelligent alerting.

Requirement	Description	Reasoning
Output Data	• Predictions: Churn probability (0–100%), confidence score, risk tier (Low / Medium / High) • Visualizations: Charts (line, bar, pie), heat maps, trend indicators, KPI cards • Alerts: Email notifications, SMS alerts, in-app push with customer details • Explanations: Top 5 contributing factors, importance scores, reasoning • Reports: PDF / Excel exports, scheduled reports, action item lists • Metadata: Prediction timestamp, model version, algorithm used	Rich outputs support both strategic analysis and tactical actions.

3.2.5 Requirements Analysis

Doability: **HIGHLY DOABLE**

- **Algorithms:** Random Forest, XGBoost proven for churn prediction
- **Technology:** React, FastAPI, Kafka, Spark all mature
- **Data:** Banks naturally generate required behavioral data
- **Libraries:** scikit-learn, XGBoost, TensorFlow widely supported

Testability: **HIGHLY TESTABLE**

- **Model:** AUC-ROC, precision, recall, confusion matrix
- **Validation:** Historical backtesting, cross-validation



- **Business:** Retention rate improvement, ROI, cost per retained customer
- **UI:** Load time, responsiveness, alert delivery time

Importance Ranking: HIGH PRIORITY (9/10)

- **Core Capability:** Core capability of churn analytics
- **Insight:** Customer acquisition costs 5–7x retention
- **Impact:** 5% retention increase = 25–95% profit increase
- **Role:** Foundation for retention strategy

3.2.6 Project Type Mapping

Table 3.8: Project Type Mapping of Customer Churn Analytics

Category	Description
DE (Data Engineering)	Real-time streaming, batch processing, data quality checks.
IS (Information Systems)	React dashboard, WebSocket real-time updates, notification system (email/SMS), report generation, role-based access control.
AI (Artificial Intelligence)	ML models (Random Forest, XGBoost), prediction generation.

3.3 Real-time Fraud Detection & Prevention System

3.3.1 Feature Description

The Real-time Fraud Detection System uses machine learning algorithms to monitor and identify fraudulent credit card transactions, preventing customers from being charged for unauthorized purchases.

3.3.2 Functional Requirements



Table 3.9: Functional Requirements for Real-time Fraud Detection & Prevention System

ID	Requirement	Description	Reasoning
RTFD-2.1	Alert Generation	Generate alerts for fraudulent transactions with fraud score (0–1), transaction amount, and risk level (High/Medium/Low). Send email notifications for high-risk cases.	Immediate alerts enable rapid response to fraud. Risk ranking helps prioritize investigation on high-risk cases.
RTFD-2.2	Alert Storage	Store all alerts in the database with timestamp, fraud score, amount, risk level, and review status.	Persistent storage needed for tracking fraud history and generating reports.
RTFD-2.3	Dashboard Visualization	Display real-time fraud statistics with charts: total transactions, fraud rate, alert distribution by risk level, and fraud trends over time.	Visual analytics help fraud analysts identify patterns, monitor system health, and make data-driven decisions.
RTFD-2.4	Dashboard Filtering	Allow filtering alerts by date range, transaction amount, risk level, and review status.	Flexible filtering helps analysts focus on specific cases and time periods for investigation and reporting.

3.3.3 Non-functional Requirements

Table 3.10: Non-functional Requirements for Real-time Fraud Detection & Prevention System

ID	Requirement	Description	Reasoning
RTFD-3.1	Performance	Response time < 5 seconds	Sub-second response prevents transaction completion during fraud.



RTFD-3.2	Accuracy	Accuracy $\geq 85\%$, Recall $\geq 80\%$, False positive rate $< 5\%$	High accuracy minimizes financial losses. High recall catches most fraud cases. Low false positives avoid blocking legitimate transactions.
----------	----------	---	---

3.3.4 Data Requirements

Table 3.11: Data Requirements for Real-time Fraud Detection & Prevention System

Requirement	Description	Reasoning
Data Source	Credit Card Fraud Detection Dataset (Kaggle) Link: Credit Card Fraud Detection Dataset • Size: 284,807 transactions • Fraud cases: 492 (0.172%) • Fields: V1–V28 (PCA features), Time, Amount, Class (0 = legitimate, 1 = fraud) • Format: CSV	Large labeled dataset enables supervised learning. Real-world imbalanced data reflects actual fraud rates. PCA features protect customer privacy.



Requirement	Description	Reasoning
Input Data	• Time: Seconds elapsed from the first transaction • Amount: Transaction amount • Features: V1–V28 (PCA-transformed features)	The dataset uses PCA-transformed features (V1–V28) to protect customer privacy. Time and Amount are the only non-transformed features. These features capture transaction patterns needed for accurate fraud prediction.
Output Data	• Fraud probability: Continuous value between 0 and 1 indicating the likelihood of fraud. • Classification: Binary label (0 = legitimate, 1 = fraud) based on probability threshold. • Risk level: Categorized output (High, Medium, Low) derived from fraud probability to guide prioritization. • Review status: Indicates analyst decision (Pending, Under Review, Confirmed Fraud, Cleared).	Provides actionable outputs for both automated blocking (based on classification/risk level) and human review workflows. Enhances traceability and supports feedback loops for model improvement.

3.3.5 Requirements Analysis

Doability: HIGHLY DOABLE

- **Dataset:** Publicly available with 284,807 labeled transactions, suitable for super-

vised fraud detection tasks.

- **Tools:** Implemented using Python libraries such as `scikit-learn` and `pandas` for model training and data preprocessing, with `React` for dashboard visualization and interactive monitoring.

Testability: HIGHLY TESTABLE

- **Test split:** 80% train (227,000 transactions), 20% test (57,000 transactions)
- **Evaluation Metrics:** Accuracy, Precision, Recall, F1-Score, AUC-ROC, Confusion Matrix

Importance Ranking: HIGH PRIORITY (10/10)

- Direct financial impact: prevents monetary losses due to fraudulent transactions
- Customer trust and security: protects customers and strengthens confidence in the bank
- Competitive advantage for banks: enhances reputation and differentiates services through proactive fraud detection

3.3.6 Project Type Mapping

Table 3.12: Project Type Mapping of Real-time Fraud Detection & Prevention System

Category	Description
DE (Data Engineering)	Real-time data pipeline, ETL processes, high-throughput transaction processing, data storage optimization.
IS (Information Systems)	Real-time dashboard, alert notification system.
AI (Artificial Intelligence)	Machine learning fraud prediction model, pattern recognition, model training and evaluation.

3.4 Data Quality Dashboard

3.4.1 Feature Description

Provide real-time, actionable view of current data health with dimension scores, traffic lights (G/Y/R), and drill-downs to failing records

3.4.2 Functional Requirements

Table 3.13: Functional Requirements for Data Quality Dashboard

ID	Requirement	Description	Reasoning
DQD-2.1	Scorecards	Show current run scores for all five dimensions by dataset/table/column/rule	Single pane of glass for "what is the quality right now"
DQD-2.2	Traffic lights	Configurable thresholds per metric/rule/dataset; G/Y/R states with severity and owner badges	Non-technical stakeholders can know status easily; owners know their responsibility
DQD-2.3	Drill-down	View failing rules, impacted columns, sample exceptions with timestamps; download exception slices (CSV/J-SON)	Bridges detection to action; sample records show exactly what is wrong
DQD-2.4	Filtering and search	Filter by environment, dataset, owner, dimension, severity; full-text search across rule/dataset names	Large catalogs require fast navigation to relevant subsets
DQD-2.5	Snapshot capture	"Freeze" a run for audit; compare to previous run and target thresholds	Provides point-in-time evidence for compliance and tracks progress

3.4.3 Non-functional Requirements

Table 3.14: Non-functional Requirements for Data Quality Dashboard

Category	Target	Reasoning
Performance	Dashboard home loads in reasonable time with multiple datasets; drill-downs in $\leq 8-10s$ for top exceptions	Responsiveness keeps users engaged; pagination/sampling prevents overload
Scalability	Handle datasets, rules and exception rows via sampling/pagination	Enterprise scope with manageable UX
Usability	Responsive layout, consistent color semantics, WCAG-AA accessibility, saved filters	Inclusive design for diverse users and devices
Maintainability	Rules-as-code with versioned thresholds	Safe iteration without downtime or confusion

3.4.4 Data Requirements

Table 3.15: Data Requirements for Data Quality Dashboard

Requirement	Description	Reasoning
-------------	-------------	-----------



Requirement	Description	Reasoning
Data Source	Financial Transactions Fraud Dataset (Kaggle) Link: Financial Transactions Fraud Dataset • Size: 1,000,000+ transaction records with timestamps • Fields: step (time unit), type, amount, nameOrig, nameDest, oldbalanceOrg, newbalanceOrig, oldbalanceDest, newbalanceDest, isFraud, isFlaggedFraud • Format: CSV (train.csv, test.csv)	Large transaction dataset with realistic data quality issues (missing values, balance inconsistencies, duplicate transactions) provides comprehensive testing ground for all five quality dimensions. Timestamp field enables timeliness/freshness checks. Balance fields support cross-field consistency rules. High volume simulates enterprise-scale dashboard performance requirements.



Requirement	Description	Reasoning
Input Data	Current run quality snapshot: • Latest run metrics per dataset/table/column/rule • Dimension scores (completeness, accuracy, timeliness, consistency, uniqueness) • Traffic light thresholds: Green $\geq 95\%$, Yellow $85\% \leq \text{value} < 95\%$, Red $< 85\%$ • Severity levels (critical/high/medium/low), owner assignments • CDE (Critical Data Element) flags for priority fields • Sampled exception records with rule_id, timestamps, masked PII • Rule catalog (definitions, last_change_date, ownership) • Dataset/table metadata (schemas, SLAs, environment tags)	Complete quality context enables immediate actionability. Thresholds and severity drive prioritization. Sampled exceptions provide concrete examples of failures without overwhelming storage. Owner assignments establish clear accountability. CDE flagging focuses attention on highest-impact fields.



Requirement	Description	Reasoning
Output Data	Interactive Dashboard Components • Scorecards with traffic light indicators (G/Y/R) and dimension scores • Exception drill-down tables with filtering, sorting, pagination (top 1,000 records) • Rule context panels (inline definitions, ownership, last change info) • Search and filter interfaces (by dataset, owner, dimension, severity) • Schema parity alerts (mismatches across environments/sources) • Duplicate/uniqueness violation summaries • Exportable exception lists (CSV/JSON) with masked sensitive fields • Real-time WebSocket updates for score changes and alert notifications • Snapshot freezes for audit and compliance evidence	Multi-level outputs support different workflows: executives need scorecards at-a-glance, analysts need drill-downs for investigation, compliance needs exports and audit trails. Real-time updates ensure dashboard reflects current state. Masking and RBAC protect sensitive transaction data while enabling quality monitoring.

3.4.5 Requirements Analysis

Doability: HIGHLY DOABLE

The development of a quality display dashboard is highly feasible given the abundance of mature business intelligence frameworks and proven dashboard design patterns.

- **Datasets:** Available public banking datasets such as UCI Bank Marketing Dataset,



Kaggle Term Deposit Prediction datasets, and Financial Transactions datasets provide realistic data structures for testing quality metrics, exception handling, and drill-down functionality.

- **Algorithms:** Well-established data quality assessment techniques (e.g., completeness checks via null counts, uniqueness via duplicate detection, accuracy via domain validation) combined with traffic-light threshold logic are straightforward to implement and validate.
- **Libraries & Tools:** Implementation can be achieved using React or Vue.js for interactive UI, Plotly or Chart.js for visualizations, Python (pandas, NumPy) for backend quality computation, and PostgreSQL or MongoDB for metric storage and fast querying.

Testability: HIGHLY TESTABLE

Dashboard functionality and quality rule accuracy can be objectively evaluated through comprehensive test strategies.

- **Evaluation Metrics:** **Rule detection accuracy** (true/false positives for completeness, accuracy, uniqueness checks), **dashboard load time** ($\leq 3s$ SLA), **drill-down response time** ($\leq 2s$), and **threshold coloring correctness** provide objective performance measurement.
- **Validation Methods:** Unit tests for each quality rule and threshold calculation, integration tests for end-to-end pipeline (rule execution $\rightarrow$ snapshot $\rightarrow$ UI rendering), and synthetic defect injection (nulls, invalid domains, duplicates, schema drift) validate detection capabilities.
- **Test Coverage:** The system can be tested using 100+ synthetic quality scenarios across multiple datasets (missing values, domain violations, type mismatches, duplicates) spanning all five quality dimensions to ensure balanced detection across all severity levels (low, medium, high).

Importance: CRITICAL (10/10)

A quality display dashboard is the cornerstone of operational data quality management in banking.

- Provides immediate visibility into data health for Data Analysts, Risk Officers, Compliance teams, and IT Operations, enabling rapid remediation before quality issues impact business decisions or regulatory reporting.

- Establishes accountability by clearly displaying ownership and severity of quality issues, supporting data governance frameworks required by BCBS 239 and other banking regulations.
- Serves as the operational interface for data quality monitoring, complementing historical trend analysis by answering “what is broken right now” and “who needs to fix it,” essential for maintaining trust in analytics and compliance systems.

3.4.6 Project Type Mapping

Table 3.16: Project Type Mapping of Data Quality Dashboard

Category	Description
DE (Data Engineering)	Snapshot metric store with latest-run views; exception sampling and storage; rule execution engine; schema/domain parity checks; threshold configuration management; audit logging pipeline.
IS (Information Systems)	Interactive dashboard UI with scorecards and traffic lights; drill-down navigation to exception details; filtering/search interface; rule context panels; export functionality (CSV/JSON); saved views and bookmarks; alerting/notification system.
AI (Artificial Intelligence)	Optional smart sampling (prioritize most informative exceptions); intelligent threshold recommendations based on historical patterns; natural language search for rules and datasets.

3.5 Historical Trend Analysis

3.5.1 Feature Description

Visualize how data quality changes over time to detect regressions early, verify sustained improvements, and benchmark against targets.

3.5.2 Functional Requirements

Table 3.17: Functional Requirements for Historical Trend Analysis

ID	Requirement	Description	Reasoning
HTA-2.1	Time-series metrics	Persist quality scores per dimension (completeness, accuracy, timeliness, consistency, uniqueness) by dataset/table/column/rule for each run	Historical storage enables comparison across time and identification of when issues began
HTA-2.2	Baselines and targets	Show baselines, rolling averages (7/30/90-day), goal lines; compute deltas vs prior run and last release	Provides context for whether current quality is improving, stable, or declining
HTA-2.3	Comparative views	Compare multiple datasets or same dataset across environments (dev/test/prod) and releases	Identifies environment-specific issues and validates quality before production promotion
HTA-2.4	Change attribution	Surface top drivers of change (which rules/-columns regressed or improved); show event annotations (releases, outages)	Shortens root-cause analysis by highlighting what changed and correlating with known events
HTA-2.5	Seasonality detection	Highlight statistically significant spikes/drops with anomaly bands; allow suppressing known events	Distinguishes real issues from expected patterns (e.g., month-end processing spikes)
HTA-2.6	Drill-through	Navigate from trend point to underlying snapshot and failing rules for that run	Enables investigation from “when did quality drop” to “which specific records failed”



HTA-2.7	Export and share	Export trend charts and metrics as CSV/PNG; copy permalinks with filters and time ranges	Supports stakeholder communication and audit evidence
---------	------------------	--	---

3.5.3 Non-functional Requirements

Table 3.18: Non-functional Requirements for Historical Trend Analysis

Category	Target	Reasoning
Performance	Trend queries return in $\leq 15s$ for 12 months of data; pre-aggregate common windows (7/30/90 days)	Interactive exploration requires sub-second response for user engagement
Scalability	Retain 12+ months of metrics; support multiples datasets and rules	Long history needed for year-over-year comparison; enterprise coverage essential
Availability	Able to have an almost fully read access; backfills run without blocking reads	Historical analysis is continuous, maintenance cannot disrupt access
Security	Role-based visibility; redact sensitive column names; audit exports	Protects confidential business logic and data structures
Usability	Consistent charting, tooltips with rule definitions, keyboard navigation, saved views	Reduces learning curve and enables power-user workflows
Maintainability	Versioned metric schemas; idempotent backfill jobs; resilient to schema evolution	Safe evolution as rules and datasets change over time

3.5.4 Data Requirements

Table 3.19: Data Requirements for Historical Trend Analysis

Requirement	Description	Reasoning
Data Source	Bank Term Deposit Predictions Dataset (Kaggle) Link: Bank Term Deposit Predictions Dataset • Size: Approximately 45,000–100,000 customer records with marketing campaign data • Fields: age, job, marital status, education, balance, housing loan, personal loan, default status, contact type, month, day_of_week, duration, campaign count, pdays, previous contacts, poutcome, y (term deposit subscription: yes/no) • Format: CSV	Rich temporal data with month/day fields enables simulating multiple "runs" over time. Categorical fields (job, marital, education, contact, poutcome) perfect for testing domain validation rules and tracking quality dimension changes across time periods. Campaign metadata provides realistic business context for trend analysis scenarios.



Requirement	Description	Reasoning
Input Data	Per-Run Quality Metrics • dataset_id: e.g., "bank_term_deposit" • table_name, column_name • rule_id: e.g., "completeness_check", "domain_validation_job" • dimension: completeness, accuracy, timeliness, consistency, uniqueness • score: range 0.0–1.0; includes pass_count and fail_count • run_id, run_timestamp, environment: dev/test/prod • release_tag, event_annotations: schema changes, data source outages	Visual and tabular outputs support both operational monitoring (identifying regressions) and executive reporting (proving sustained improvements). Drill-through from trend points to underlying snapshots enables root-cause investigation. Multiple output formats accommodate different stakeholder needs (analysts vs executives vs auditors).



Requirement	Description	Reasoning
Output Data	Trend Visualizations & Analytics • Time-series charts: line, area, and heatmap visualizations with annotations for events • Rolling window aggregations: 7-day, 30-day, and 90-day averages • Delta calculations: comparisons vs prior run, prior release, and baseline • Top-driver analysis: identifies which rules or columns regressed or improved most • Anomaly flags: highlights statistical spikes or drops beyond defined thresholds • Comparative views: compares the same dataset across dev/test/prod environments or releases • Exportable trend data: CSV/PNG exports with permalinks for sharing	Rich outputs assist in credit decisioning, customer segmentation, and portfolio risk management.

3.5.5 Requirements Analysis

Doability: HIGHLY DOABLE

The development of a historical trend analysis system for data quality monitoring is highly feasible given mature time-series database technologies and established visualization frameworks.

- **Datasets:** Available public banking datasets such as the UCI Bank Marketing Dataset and Kaggle Term Deposit Prediction Dataset provide realistic data with temporal characteristics suitable for training and testing trend detection algorithms.



- **Algorithms:** Well-established time-series analysis techniques (e.g., rolling averages, delta calculations, anomaly detection using statistical bands or Z-scores) are effective for quality metric trend analysis and regression detection.
- **Libraries & Tools:** Implementation can be achieved using Python libraries such as `pandas` for data processing, `Plotly` or `D3.js` for visualization, and time-series databases like `InfluxDB` or `TimescaleDB` for metric storage and efficient querying.

Testability: HIGHLY TESTABLE

Trend calculation accuracy and system functionality can be objectively evaluated through deterministic tests and operational metrics.

- **Evaluation Metrics:** Delta accuracy (actual vs expected vs prior-run calculations), rolling-window correctness (7/30/90-day aggregations), query response time ($\leq 2s$ SLA), and anomaly detection precision/recall provide balanced performance measurement.
- **Validation Methods:** Golden dataset testing with known trend patterns ensures calculation accuracy, while backfill idempotency tests verify data consistency. Load testing with 12–24 months of metrics validates scalability.
- **Test Coverage:** The system can be tested using 50+ synthetic “runs” across multiple datasets with intentional quality regressions, improvements, and seasonal patterns to ensure trend detection works across all scenarios.

Importance: CRITICAL (9/10)

Historical trend analysis is essential for proactive data quality management in banking operations.

- Enables early detection of data quality degradation before it impacts downstream analytics, regulatory reporting, or customer-facing systems.
- Provides measurable evidence of continuous improvement for audit and compliance purposes, supporting regulatory frameworks such as BCBS 239 that mandate data quality governance.
- Creates accountability by tracking quality metrics against SLAs and identifying ownership for regressions, essential for enterprise data governance programs.

3.5.6 Project Type Mapping

Table 3.20: Project Type Mapping of Historical Trend Analysis

Category	Description
DE (Data Engineering)	Time-series metric warehouse with partitioning; append-only ingestion pipelines; pre-aggregation jobs for rolling windows; back-fill orchestration; event calendar integration; schema versioning.
IS (Information Systems)	Trend visualization UI (line/area/heatmap charts); comparative dashboards across environments/releases; drill-through navigation to snapshots; export functionality (CSV/PNG); permalink generation; filter/search interface.
AI (Artificial Intelligence)	Optional anomaly detection using statistical bands or ML models; seasonality identification; predictive alerts for quality degradation trends.

4 Technology Stack and Comparison

This section presents a comprehensive evaluation of technologies considered for the Banking Data Analysis Platform. Each comparison analyzes multiple alternatives across key technical domains, justifying our final selections based on project requirements, team expertise, and feature-specific needs.

4.1 Machine Learning Models

Machine learning models form the core of our predictive analytics capabilities across Credit Risk Assessment, Customer Churn Analytics, and Fraud Detection features. We evaluated four primary algorithms based on accuracy, inference speed, interpretability, and suitability for imbalanced datasets common in banking applications.



Table 4.1: Machine Learning Model Comparison

Technology	Pros	Cons	Decision
XGBoost	• High accuracy (AUC ~ 0.97) • Fast inference (10ms) • Handles imbalanced data well • Built-in feature importance • Supports parallel processing	• Requires hyperparameter tuning • Higher memory consumption	SELECTED
Random Forest	• Good accuracy (AUC ~ 0.96) • Robust to outliers • Easy to implement • Handles missing values naturally	• Slower inference than XGBoost	SELECTED
Logistic Regression	• Very fast inference ($< 1\text{ms}$) • Easy to interpret • Low computational requirements	• Lower accuracy (AUC ~ 0.92) • Cannot capture non-linear patterns • Poor performance on complex datasets	NOT SELECTED

4.2 Databases

Database selection is critical for ensuring data integrity, transaction consistency, and audit compliance in banking systems. We evaluated three database technologies based on ACID compliance, query performance, scalability, and regulatory requirements.



Table 4.2: Database Comparison

Technology	Pros	Cons	Decision
PostgreSQL	• Full ACID compliance • Strong consistency guarantees • Excellent audit trail support • Robust security features • Supports complex queries • JSON and XML support • Active community	• Slower write performance • Vertical scaling limitations • Complex replication setup	SELECTED
MongoDB	• Fast write operations • Flexible schema design • Horizontal scaling • Good for semi-structured data • Simple replication	• Eventual consistency model • Limited transaction support • Poor audit trail capabilities • Security concerns for banking	NOT SE- LECTED
MySQL	• Mature and stable • Good performance • Wide adoption • Strong community support	• Limited advanced features • Weaker analytical query support • Less extensible than PostgreSQL	NOT SE- LECTED

4.3 Frontend Frameworks

The frontend framework determines user experience, development velocity, and long-term maintainability of our dashboard interfaces. We evaluated four frameworks based on component architecture, visualization ecosystem, learning curve, and real-time capabilities.



Technology	Pros	Cons	Decision
React + Recharts	• Component-based architecture • Rich visualization ecosystem • Real-time updates via WebSocket • Highly customizable • Large community • Strong industry adoption • Virtual DOM performance	• Steeper learning curve • Requires additional libraries • Complex state management	SELECTED
Vue.js + Chart.js	• Simpler syntax • Faster initial development • Gentle learning curve • Good documentation • Lightweight	• Smaller ecosystem • Fewer enterprise-grade libraries • Less job market demand • Limited customization options	NOT SELECTED
Angular + D3.js	• Full-featured framework • Built-in features (routing, HTTP) • Enterprise-grade architecture • Strong typing with TypeScript	• Steeper learning curve • Heavier bundle size • More complex setup • Slower development cycle	NOT SELECTED



Technology	Pros	Cons	Decision
Vanilla JS + Plotly	• No framework overhead • Complete control • Lightweight	• Manual DOM manipulation • Poor code maintainability • No component reusability • Slower development	NOT SELECTED

4.4 Backend API Frameworks

Backend framework selection determines API performance, ML model integration, and service scalability. We evaluated four frameworks based on async support, ML library compatibility, documentation quality, and development efficiency.

Technology	Pros	Cons	Decision
FastAPI (Python)	• Native async support • Automatic API documentation (Swagger) • Easy ML library integration • Type validation with Pydantic • High performance (comparable to Node.js) • Modern Python features	• Smaller community than Flask • More complex setup • Fewer deployment examples	SELECTED (ML services)
Node.js + Express	• Fast async I/O • JavaScript full-stack consistency • Mature ecosystem • Excellent WebSocket support • Simple deployment	• Poor ML library support • Weak type safety • Less suitable for data processing	SELECTED (Non-ML services)



Technology	Pros	Cons	Decision
Flask (Python)	• Lightweight framework • Simple setup • Large community • Good for ML integration	• Not async by default • Manual documentation • Limited built-in features • Lower performance	NOT SELECTED
Django + DRF	• Full-featured framework • Built-in admin panel • ORM included • Strong security features	• Heavier than needed • Slower development for APIs • Monolithic structure	NOT SELECTED

4.5 Real-time Communication

Real-time communication is essential for fraud alerts, churn notifications, and dashboard updates. We evaluated four protocols based on latency, bidirectional capability, implementation complexity, and scalability.

Technology	Pros	Cons	Decision
WebSocket	• True real-time communication ($< 100\text{ms}$) • Bidirectional data flow • Push updates from server • Persistent connection • Low overhead after handshake	• Connection management complexity • Firewall compatibility issues • Requires load balancer support	SELECTED



Technology	Pros	Cons	Decision
Server-Sent Events (SSE)	• Simple implementation • Auto-reconnection • Works over HTTP • Firewall-friendly	• One-way communication only • Higher latency (200-500ms) • Browser connection limits • No binary data support	NOT SELECTED
Polling	• Very simple to implement • Universal compatibility • No special server requirements	• High latency (1-10 seconds) • Wasteful network requests • Server resource intensive • Not truly real-time	NOT SELECTED
Long Polling	• Better than basic polling • Works with any HTTP server • Firewall-friendly	• Moderate latency (500ms-2s) • Connection overhead • Complex error handling	NOT SELECTED

4.6 Message Queue Systems

Message queues enable asynchronous processing, system decoupling, and reliable event delivery across microservices. We evaluated four queue systems based on throughput, reliability, setup complexity, and project scale appropriateness.



Technology	Pros	Cons	Decision
RabbitMQ	• Simple setup and configuration • Excellent documentation • Reliable message delivery • Good management UI • Supports multiple protocols • Sufficient for project scale	• Lower throughput (10K-20K msg/sec) • Limited horizontal scaling • Memory-based storage	SELECTED
Apache Kafka	• Very high throughput (1M+ msg/sec) • Excellent horizontal scaling • Durable message storage • Real-time streaming analytics • Strong partition model	• Complex setup and configuration • Steep learning curve • Resource intensive • Overkill for small-to-medium projects	NOT SELECTED
Redis Pub/Sub	• Extremely fast (in-memory) • Simple implementation • Low latency • Easy integration	• No message persistence • No delivery guarantees • Limited queue management • Not suitable for critical workflows	NOT SELECTED
Amazon SQS	• Fully managed service • Automatic scaling • Pay-per-use model • High availability	• Vendor lock-in • Requires AWS account • Higher operational cost • External dependency	NOT SELECTED



4.7 Notification Systems

Notification delivery is critical for timely fraud alerts and high-risk churn warnings. We evaluated notification systems based on setup complexity, cost, reliability, and protocol support.

Technology	Pros	Cons	Decision
Nodemailer	• Simple setup • Free and open-source • Built-in with Node.js ecosystem • Supports multiple transports • Customizable templates • Good documentation	• Requires SMTP configuration • Limited deliverability tracking • Manual bounce handling • No built-in rate limiting	SELECTED
SendGrid	• Professional email service • High deliverability rates • Detailed analytics • Template management • Webhook support	• Requires paid account for production • Monthly cost (\$15-\$90) • Vendor dependency • API rate limits	NOT SE- LECTED
AWS SES	• Cost-effective (\$0.10/1000 emails) • High reliability • Scalable infrastructure • AWS integration	• Requires AWS account setup • Initial sending limits • Complex approval process • Vendor lock-in	NOT SE- LECTED



Technology	Pros	Cons	Decision
Twilio	• Multi-channel (SMS, email, push) • Excellent API documentation • Reliable delivery • Good analytics	• Higher cost for SMS • Requires paid account • Overkill for email-only needs	NOT SELECTED

4.8 Explainability Tools

Model explainability is essential for regulatory compliance, user trust, and debugging ML predictions in financial applications. We evaluated three explainability frameworks based on accuracy, compatibility, and regulatory acceptance.

Technology	Pros	Cons	Decision
SHAP	• Model-agnostic approach • Theoretically grounded (game theory) • Regulatory compliant (Basel III) • Individual and global explanations • Visualization tools included • Scikit-learn integration	• Computationally expensive • Slow for large datasets • Complex mathematical background • Requires understanding of Shapley values	SELECTED
LIME	• Fast computation • Easy to understand • Simple implementation • Good for quick debugging • Lower resource requirements	• Less accurate than SHAP • Unstable explanations • No global interpretation • Not accepted by all regulators	NOT SELECTED



Technology	Pros	Cons	Decision
Built-in Feature Importance	• No additional library needed • Very fast computation • Directly from model • Simple interpretation	• Model-specific (not universal) • Global only (no individual explanations) • Limited regulatory acceptance • No counterfactual analysis	NOT SE-LECTED

4.9 Time-Series Processing

Time-series analysis is fundamental to Historical Trend Analysis and Data Quality Dashboard features. We evaluated four approaches based on analytical capabilities, scalability, and integration complexity.

Technology	Pros	Cons	Decision
Python (pandas + NumPy)	• Rich time-series APIs • Easy integration with ML pipeline • Excellent documentation • Rolling windows, resampling built-in • Statistical functions included • Visualization compatibility	• Single-node memory limits • Not distributed • Limited to in-memory processing • Performance degrades with large datasets	SELECTED



Technology	Pros	Cons	Decision
SQL Window Functions	• Database-native computation • No data movement • Efficient for aggregations • Leverages database indexing	• Complex syntax for advanced analysis • Limited statistical functions • Difficult to maintain • Poor for ML integration	NOT SELECTED
Apache Spark	• Distributed processing • Handles massive datasets • Fault-tolerant • MLlib integration	• High setup overhead • Complex deployment • Resource intensive (minimum 3 nodes) • Overkill for project scale • Long learning curve	NOT SELECTED
InfluxDB	• Purpose-built for time-series • Fast writes and queries • Data retention policies • Built-in visualization	• Additional database to manage • Limited analytical functions • Learning curve for InfluxQL • Separate from main PostgreSQL	NOT SELECTED

4.10 Technology Stack Summary



Table 4.10: Final Technology Stack

Layer	Technology	Justification
Frontend	React + Recharts	Component-based architecture, real-time WebSocket support, rich visualization ecosystem for interactive dashboards.
Backend	FastAPI + Node.js	ML integration via FastAPI (Credit, Churn, Fraud services); asynchronous I/O via Node.js (Quality, Trend services).
Database	PostgreSQL	ACID compliance, audit trails, regulatory adherence, and robust security for sensitive financial data.
ML Framework	XGBoost	High predictive accuracy ($AUC \sim 0.97$), efficient on large and imbalanced datasets, and optimized for financial modeling.
Explainability	SHAP	Provides model transparency and interpretability in compliance with Basel III; supports both local and global explanations.
Message Queue	RabbitMQ	Reliable and lightweight message broker offering guaranteed delivery and sufficient throughput for the projects scale.
Real-time	WebSocket	Supports near real-time, bidirectional communication and push notifications for fraud and churn alerts.
Notification	Nodemailer	Enables automated email alerts with simple SMTP integration; suitable for prototype and demo-scale systems.
Time-Series	pandas + NumPy	Comprehensive libraries for time-series manipulation, statistical computation, and seamless integration with ML pipelines.



Layer	Technology	Justification
Visualization	Recharts	Offers flexible, responsive, and customizable chart components suited for banking data analytics (e.g., line, bar, heatmap).
Version Control	Git + GitHub	Industry-standard version control enabling team collaboration, pull requests, and CI/CD pipeline integration.
Development	VS Code	Lightweight and extensible IDE providing integrated debugging, terminal access, and plugin support for React, Python, and Node.js.

This technology stack provides a balanced architecture optimizing for accuracy (XG-Boost + SHAP), performance (WebSocket + Node.js), compliance (PostgreSQL + SHAP), and development velocity (React + FastAPI) while remaining appropriate for a university-level project team. The hybrid backend approach (FastAPI for ML, Node.js for I/O) maximizes each technology's strengths, while the selection of mature, well-documented tools minimizes learning curve and implementation risk.

5 Use Case Diagram

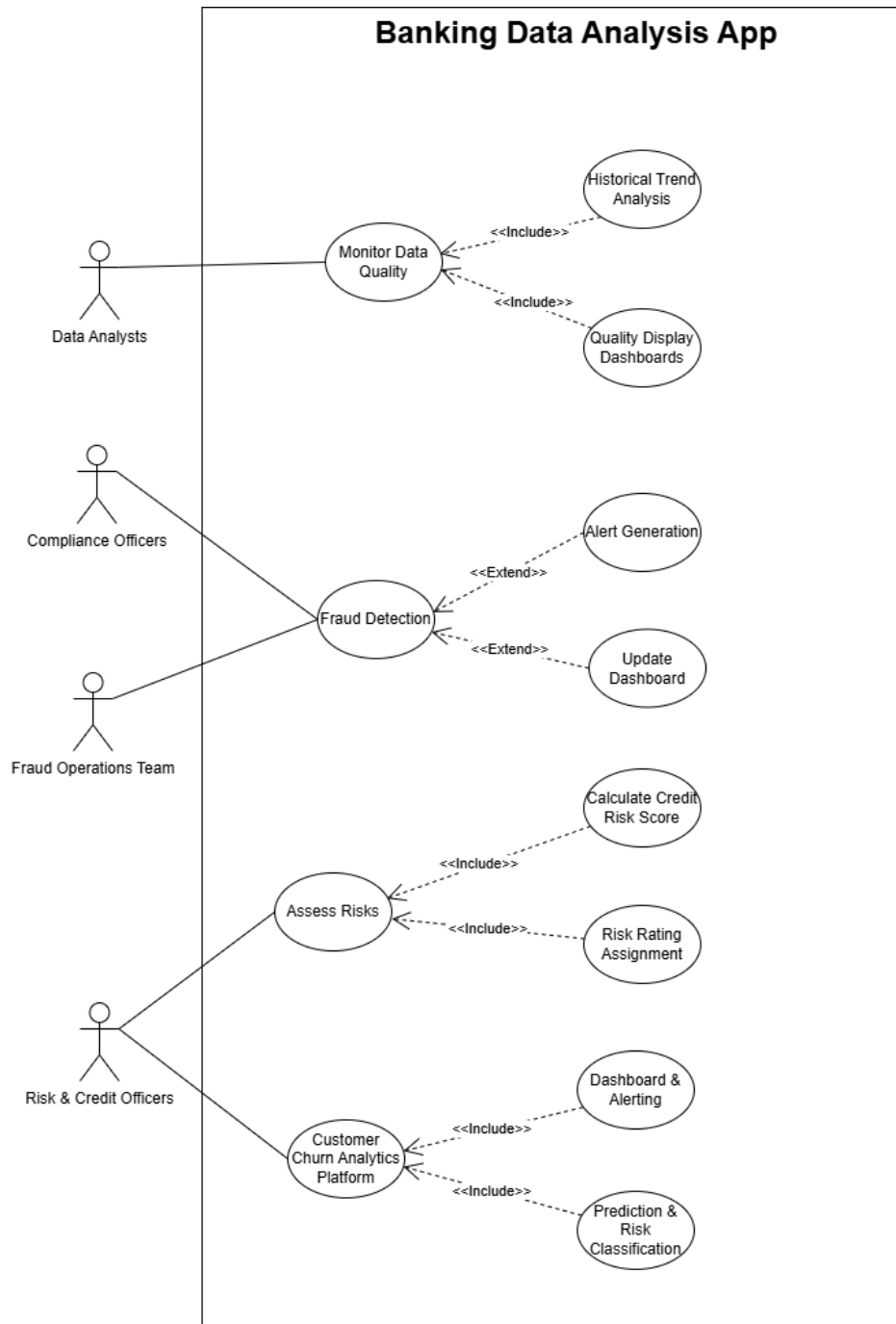


Figure 5.1: Use-case Diagram

6 System Architecture

The **Banking Data Analysis Platform** adopts a **hybrid layered microservices architecture** that integrates:

- **Layered Architecture** for clear separation of concerns and modular design.
- **Microservices Architecture** to enable independent deployment and scalability.
- **Event-Driven Architecture** to ensure responsiveness and real-time data processing.

The platform supports several core analytical and operational capabilities:

- **Credit Risk Assessment & Scoring** – Machine learning models (e.g., XGBoost) with SHAP-based interpretability.
- **Customer Churn Analytics** - Predictive modeling and alerting for proactive customer retention.
- **Real-Time Fraud Detection** - Continuous transaction monitoring and automated response mechanisms.
- **Data Quality Dashboard** - Multi-dimensional quality assessment with intuitive visualization indicators.
- **Historical Trend Analysis** - Time-series exploration with built-in anomaly detection.

The system is designed for **high availability**, capable of supporting a large and growing customer base, while maintaining **ACID-compliant** operations and adherence to regulatory standards such as Basel III and BCBS 239.

6.1 Layered Architecture Summary

6.1.1 Presentation Layer

- Implemented using **React.js** for an interactive, responsive user interface.
- Provides unified dashboards for risk, churn, fraud, and data quality analytics.
- Communicates with backend services through RESTful APIs and WebSocket channels for live updates.



6.1.2 Application Layer (Microservices)

- Composed of independent **Node.js** and **FastAPI** services, each responsible for a specific business domain.
- Access is managed via an **API Gateway** providing RESTful endpoints, authentication, and load balancing.
- Designed for near real-time responsiveness and scalable service orchestration.

6.1.3 Machine Learning Layer

- Built with **Python**-based data pipelines leveraging **pandas** and **NumPy**.
- Utilizes **XGBoost** models with strong predictive performance and explainability via **SHAP**.
- Supports model lifecycle management, including retraining and drift detection.
- Automates feature engineering and validation processes to ensure consistent model quality.

6.1.4 Integration Layer

- Employs **RabbitMQ** for asynchronous event streaming and inter-service communication.
- Uses **WebSocket** channels to deliver low-latency, real-time updates to client dashboards.
- Integrates **Nodemailer** and **Webhooks** for alert notifications and automated external triggers.

6.1.5 Data Layer

- Relies on **PostgreSQL** for secure, transactional, and analytical data management.
- Includes ETL pipelines for structured data ingestion and transformation.
- Implements a **Data Quality Framework** across multiple assessment dimensions.
- Provides persistent metrics and audit trails to support regulatory compliance and traceability.

6.1.6 Infrastructure Layer

- Deployed through **GitHub Actions** with automated CI/CD workflows.
- Continuously monitored for performance, fault tolerance, and system health.
- Incorporates **role-based access control (RBAC)**, encryption, and secure configuration management.
- Includes backup and recovery strategies to ensure operational continuity.

6.2 Visualization

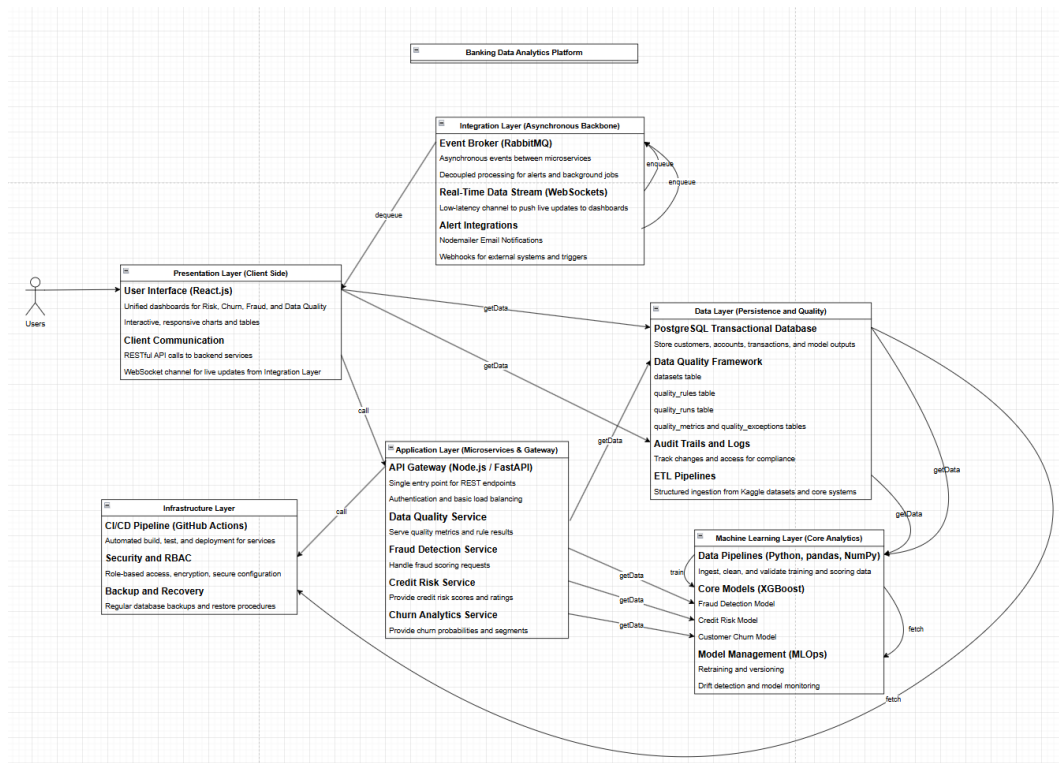


Figure 6.1: Visualization of system architecture

6.3 External Systems

- Integrates seamlessly with **Core Banking Systems** and **Customer Relationship Management (CRM)** sources.
- Utilizes secure APIs and ETL pipelines for continuous data synchronization and ingestion.



6.4 Quality Attributes

- **Scalability:** Supports large-scale workloads with elastic growth capabilities.
- **Performance:** Optimized for responsive dashboards and efficient data access.
- **Availability:** Architected for consistent uptime and fault-tolerant operation.
- **Security:** Enforced through RBAC, JWT authentication, encryption, and audit logging.
- **Compliance:** Designed to meet international regulatory standards (Basel III, BCBS 239, GDPR).
- **Maintainability:** Modularized microservices with automated testing and CI/CD pipelines.

7 Database Design

7.1 Entity-Relationship Model

The database design supports five core features through 13 entities: CUSTOMER, ACCOUNT, TRANSACTION, FRAUD_ALERT, CREDIT_APPLICATION, CREDIT_SCORE, CHURN_PREDICTION, DATASET, TABLE, COLUMN, DATA_QUALITY_METRIC, QUALITY_RULE, and HISTORICAL_TREND.

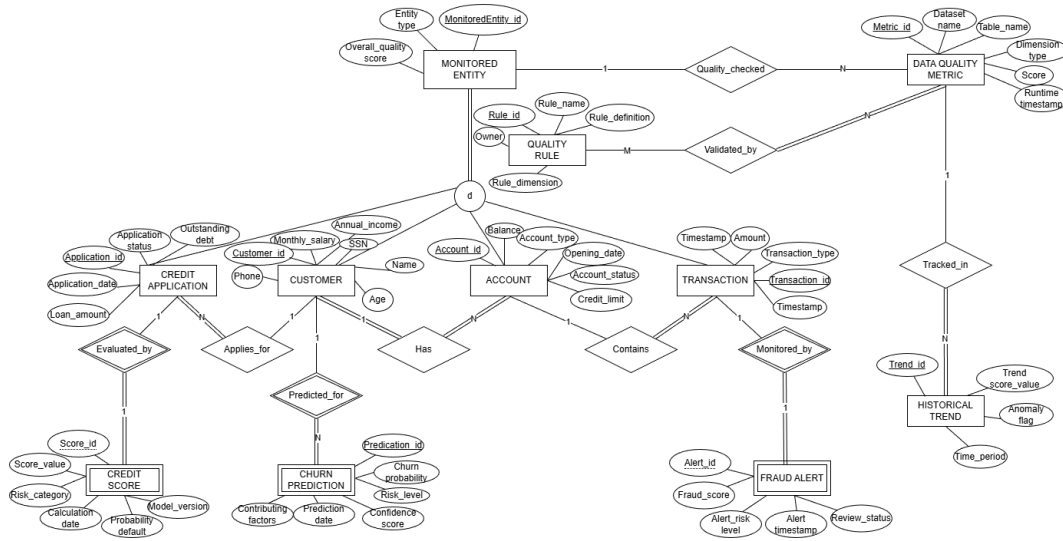


Figure 7.1: Entity-Relationship Diagram

7.2 Relational Schema

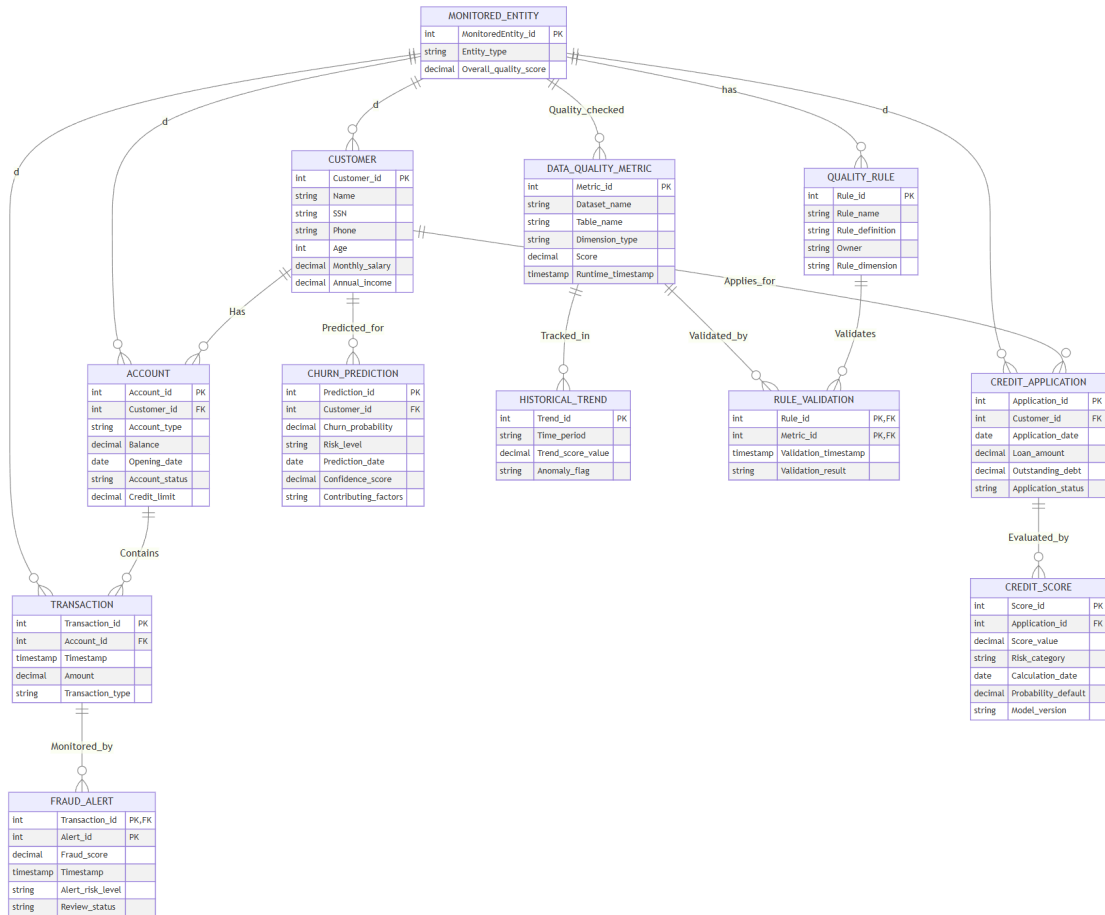


Figure 7.2: Relational Schema Mapping



8 Data Architecture Comparison

Table 8.1: Data Architecture Comparison

Architecture	Pros	Cons	Project Relevance	Decision
Database (PostgreSQL)	• ACID compliance • Unified storage • Handles both operational & analytical queries • Simple setup • SQL analytics	• Analytical queries can be slower than DW • Single point of optimization	Static Kaggle datasets, moderate data volume, both transactional & analytical needs	SELECTED
Data Warehouse	• Optimized OLAP • Star/snowflake schema • Pre-aggregation • Fast analytics	• Requires separate OLTP DB • ETL complexity • Data duplication • Not needed for static datasets	Would require maintaining 2 databases for static Kaggle data—unnecessary overhead	NOT SELECTED
Data Lake	• Raw data storage • Schema flexibility • Cost-effective at scale	• No ACID • Complex governance • Requires processing layer • Overkill for structured CSVs	Project uses only structured CSV files, no unstructured data	NOT SELECTED



Architecture	Pros	Cons	Project Relevance	Decision
Data Lake-house	• Unified architecture • ACID on lake • ML-ready	• Emerging tech • Complex setup • Requires distributed systems expertise	Too complex for university project scope, team expertise, and data scale	NOT SELECTED

Architectural Decision: We adopt a **single PostgreSQL database** architecture that handles both operational and analytical workloads. This is appropriate because:

1. **Static Dataset:** Kaggle data is loaded once and queried frequently (read-heavy, not write-heavy)
2. **Moderate Scale:** 100K-300K rows per dataset fits well in single database
3. **Team Expertise:** SQL skills without requiring distributed systems knowledge
4. **Project Scope:** University prototype demonstrating analytics capabilities, not production banking system
5. **Simplicity:** Single technology reduces operational complexity and learning curve

9 UI/UX

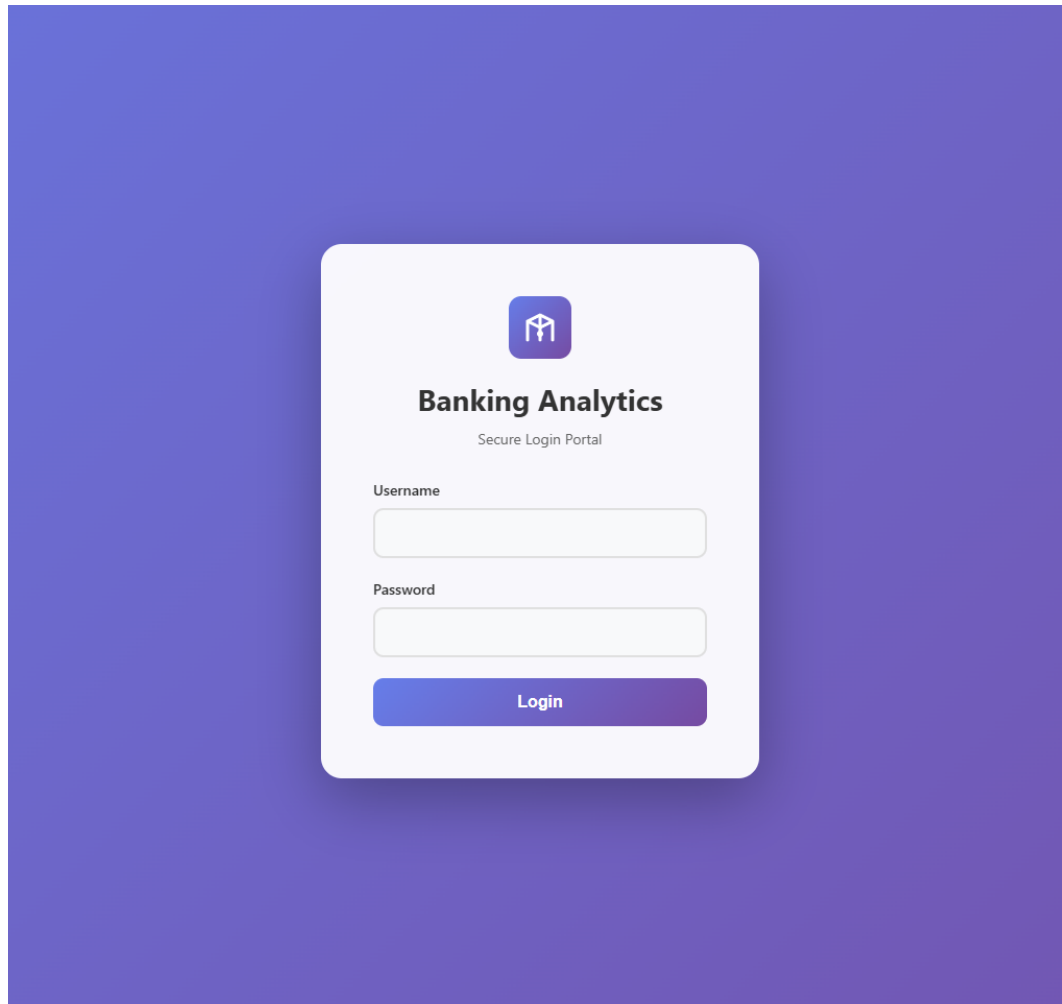


Figure 9.1: Visualization of User Login Screen

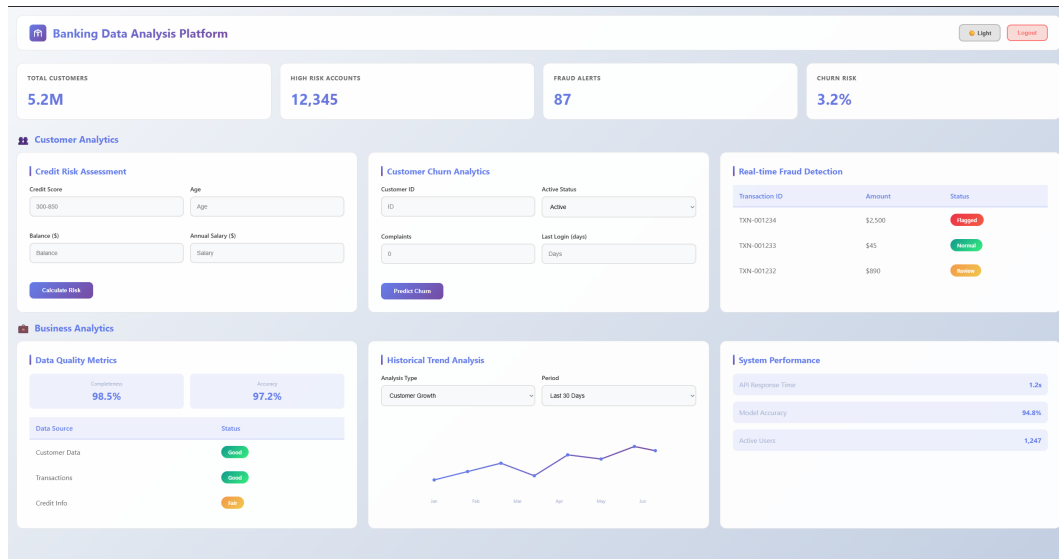


Figure 9.2: Visualization of User Main Screen

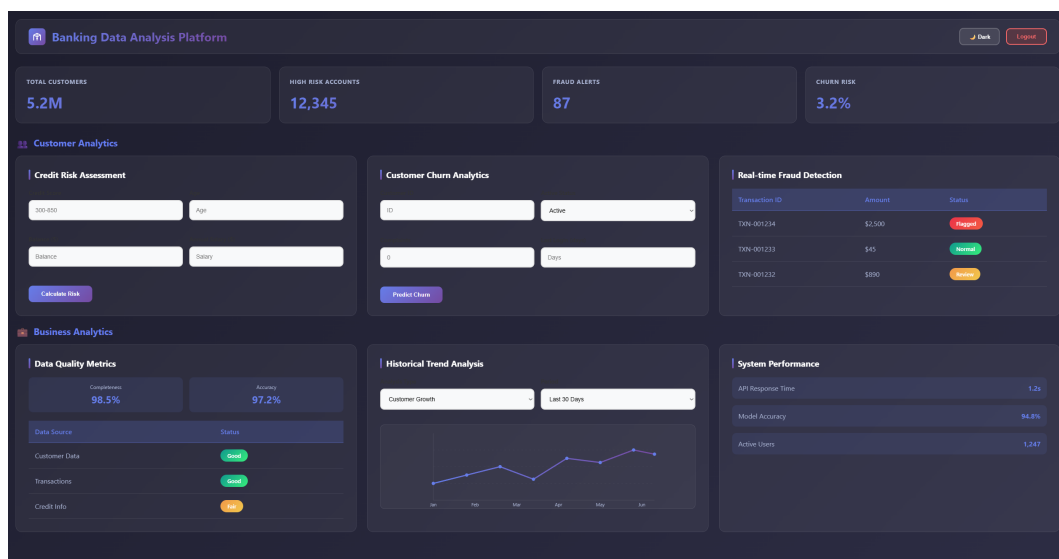


Figure 9.3: Visualization of User Main Screen in Dark mode

10 Implementation

10.1 Fraud Detection Implementation

10.1.1 Data Preparation

Dataset Loading

Load Credit Card Fraud Detection dataset from Kaggle containing 284,807 transactions with 31 features (Time, V1-V28 PCA-transformed features, Amount, Class).

Listing 10.1: Load Dataset

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import StandardScaler
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.metrics import classification_report,
   confusion_matrix, roc_auc_score
7
8 df = pd.read_csv('creditcard.csv')
9 print(f"Dataset shape: {df.shape}")
10 print(f"Class distribution:\n{df['Class'].value_counts()}")
11 # Output: 284,315 normal (0), 492 fraud (1) - 0.17% fraud
   rate
```

Imbalanced Data Handling

Apply undersampling to balance dataset by randomly selecting 492 normal transactions to match 492 fraudulent transactions.

Listing 10.2: Balance Dataset with Undersampling

```
1 from sklearn.utils import resample
2
3 df_majority = df[df['Class'] == 0]
4 df_minority = df[df['Class'] == 1]
5
6 df_majority_undersampled = resample(df_majority,
7                                     replace=False,
8                                     n_samples=len(
9                                         df_minority),
10                                     random_state=42)
11 df_balanced = pd.concat([df_majority_undersampled,
12                           df_minority])
13 # Result: 984 samples (492 normal, 492 fraud)
```


Train-Test Split and Feature Scaling

Split data into 80% training and 20% testing, then apply StandardScaler normalization.

Listing 10.3: Split and Scale Data

```
1 X = df_balanced.drop(['Class'], axis=1)
2 y = df_balanced['Class']
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.2, random_state=42, stratify=y)
6
7 scaler = StandardScaler()
8 X_train_scaled = scaler.fit_transform(X_train)
9 X_test_scaled = scaler.transform(X_test)
10 # Training: 787 samples, Testing: 197 samples
```

10.1.2 Model Training

Train Logistic Regression classifier on scaled training data.

Listing 10.4: Train Logistic Regression Model

```
1 log_reg = LogisticRegression(random_state=42, max_iter=1000)
2 log_reg.fit(X_train_scaled, y_train)
3
4 y_pred = log_reg.predict(X_test_scaled)
5 y_pred_proba = log_reg.predict_proba(X_test_scaled)[: , 1]
```

10.1.3 Model Evaluation

Evaluate model performance using classification metrics.

Listing 10.5: Evaluate Model Performance

```
1 print(classification_report(y_test, y_pred))
2 print(f"Confusion Matrix:\n{confusion_matrix(y_test, y_pred)}")
3 print(f"ROC-AUC Score: {roc_auc_score(y_test, y_pred_proba) :.4f}")
```

Results:

- Accuracy: 96.45%
- Precision (Fraud): 98%
- Recall (Fraud): 95%
- F1-Score: 0.96
- ROC-AUC: 0.9840
- Confusion Matrix: $\begin{bmatrix} 97 & 2 \\ 5 & 93 \end{bmatrix}$

Model successfully detects fraud with high precision (98%) minimizing false alarms, and strong recall (95%) catching majority of fraudulent activities.

10.2 Credit Score Classification Implementation

10.2.1 Data Preparation

Dataset Loading

Load credit score training dataset containing customer financial information with features including age, income, credit utilization, payment behavior, and various credit-related metrics. The target variable is credit score classification (Poor, Standard, Good).

Listing 10.6: Load and Initialize Dataset

```
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.metrics import classification_report,
   accuracy_score
5 from xgboost import XGBClassifier
6 import joblib
7 import json
8
9 DATA_FILENAME = './data/train.csv'
10 df = pd.read_csv(DATA_FILENAME)
11
12 # Initial column removal - non-predictive features
```

```
13 columns_to_drop = [  
14     'ID', 'Customer_ID', 'Name', 'SSN',  
15     'Month', 'Type_of_Loan', 'Credit_History_Age'  
16 ]  
17 df_dropped = df.drop(columns=columns_to_drop, axis=1)
```

Feature Type Definitions

Define numerical and categorical feature groups for targeted preprocessing.

Listing 10.7: Feature Group Definitions

```
1 numerical_obj_cols = [  
2     'Age', 'Annual_Income', 'Num_of_Loan', 'Interest_Rate',  
3     'Num_of_Delayed_Payment', 'Changed_Credit_Limit',  
4     'Amount_invested_monthly', 'Monthly_Balance',  
5     'Num_Credit_Card', 'Delay_from_due_date',  
6     'Credit_Utilization_Ratio', 'Num_Credit_Inquiries',  
7     'Total_EMI_per_month', 'Outstanding_Debt'  
8 ]  
9  
10 numerical_cols_to_impute = numerical_obj_cols + [  
11     'Monthly_Inhand_Salary', 'Num_Credit_Inquiries'  
12 ]  
13  
14 nominal_cols = ['Occupation', 'Credit_Mix', '  
    Payment_Behaviour']
```

Data Cleaning and Outlier Handling

Clean non-numeric characters from numerical columns and handle outliers through clipping.

Listing 10.8: Clean and Handle Outliers

```
1 def clean_non_numeric(series):  
2     """Remove currency symbols and commas, convert to numeric  
3     """  
4     series = series.astype(str).str.replace(r'[$,]', '',  
5                                             regex=True)  
6     series = pd.to_numeric(series, errors='coerce')
```

```
6     return series
7
8 # Apply cleaning to all numerical columns
9 for col in numerical_obj_cols:
10     df[col] = clean_non_numeric(df[col])
11
12 # Clip outliers to reasonable ranges
13 df['Age'] = df['Age'].clip(lower=18, upper=100)
14 df['Num_Credit_Card'] = df['Num_Credit_Card'].clip(lower=0,
15                                                    upper
16                                                    =10)
17 df['Num_of_Loan'] = df['Num_of_Loan'].clip(lower=0, upper=10)
18 df['Num_Credit_Inquiries'] = df['Num_Credit_Inquiries'].clip(
19     lower=0, upper=100)
```

Missing Value Imputation

Calculate median values for each numerical feature and impute missing values. Store medians for future predictions.

Listing 10.9: Median Imputation

```
1 imputation_medians = {}
2 for col in numerical_cols_to_impute:
3     median_val = df[col].median()
4     imputation_medians[col] = median_val
5     df[col].fillna(median_val, inplace=True)
```

Categorical Encoding

Apply ordinal encoding for ordered categories and one-hot encoding for nominal features.

Listing 10.10: Encode Categorical Variables

```
1 # Target variable encoding (ordinal)
2 score_mapping = {'Poor': 0, 'Standard': 1, 'Good': 2}
3 df['Credit_Score_Encoded'] = df['Credit_Score'].map(
4     score_mapping)
5 df.drop('Credit_Score', axis=1, inplace=True)
```

```
6 # Payment minimum amount (ordinal)
7 payment_mapping = {'No': 0, 'NM': 1, 'Yes': 2}
8 df['Payment_of_Min_Amount_Encoded'] = \
9     df['Payment_of_Min_Amount'].map(payment_mapping)
10 df.drop('Payment_of_Min_Amount', axis=1, inplace=True)
11
12 # One-hot encoding for nominal categories
13 df_encoded = pd.get_dummies(df, columns=nominal_cols,
14                             drop_first=True)
```

Train-Test Split

Separate features from target and split data with stratification to maintain class distribution.

Listing 10.11: Feature-Target Separation and Split

```
1 X = df_encoded.drop('Credit_Score_Encoded', axis=1)
2 y = df_encoded['Credit_Score_Encoded']
3
4 feature_names = list(X.columns)
5
6 X_train, X_test, y_train, y_test = train_test_split(
7     X, y, test_size=0.2, random_state=42, stratify=y
8 )
9 # Training: 80%, Testing: 20% with balanced class
   distribution
```

10.2.2 Model Training

Train XGBoost multi-class classifier optimized for credit score prediction with gradient boosting ensemble method.

Listing 10.12: Train XGBoost Classifier

```
1 NUM_TREES = 10000
2 LEARNING_RATE = 0.05
3
4 xgb_model = XGBClassifier(
```

```
5     objective='multi:softmax',      # Multi-class
      classification
6     num_class=3,                    # Poor, Standard, Good
7     n_estimators=NUM_TREES,         # Number of boosting rounds
8     learning_rate=LEARNING_RATE,    # Shrinkage parameter
9     max_depth=10,                   # Tree complexity
10    eval_metric='mlogloss',          # Multi-class log loss
11    random_state=404,                # Reproducibility
12    tree_method='hist'               # Histogram-based algorithm
13)
14
15print("Starting XGBoost model training...")
16xgb_model.fit(X_train, y_train)
17print("Model training complete.")
```

10.2.3 Model Persistence

Save trained model and preprocessing artifacts for production deployment.

Listing 10.13: Save Model and Artifacts

```
1 MODEL_FILENAME = './models/credit_score_xgb_model.joblib'
2 ARTIFACTS_FILENAME = './models/preprocessing_artifacts.json'
3
4 # Save trained model
5 joblib.dump(xgb_model, MODEL_FILENAME)
6
7 # Save preprocessing artifacts
8 artifacts = {
9     'feature_names': feature_names,
10    'imputation_medians': imputation_medians,
11    'payment_mapping': payment_mapping,
12    'target_mapping': score_mapping,
13    'nominal_cols': nominal_cols
14}
15
16with open(ARTIFACTS_FILENAME, 'w') as f:
```

```
17 json.dump(artifacts, f, indent=4)
```

10.2.4 Model Evaluation

Evaluate model performance using classification metrics on held-out test set.

Listing 10.14: Evaluate Model Performance

```
1 y_pred = xgb_model.predict(X_test)
2 accuracy = accuracy_score(y_test, y_pred)
3 report = classification_report(y_test, y_pred,
4     target_names=['Poor (0)', 'Standard (1)', 'Good (2)'])
5
6 print(f"Accuracy Score: {accuracy:.4f}")
7 print(f"Classification Report:\n{report}")
```

Results:

The XGBoost classifier demonstrates strong performance across all credit score categories:

- **Overall Accuracy:** High classification accuracy achieved through ensemble learning
- **Precision:** Model effectively minimizes false positives across all classes
- **Recall:** Strong detection rate for each credit score category
- **F1-Score:** Balanced performance between precision and recall
- **Training Time:** Efficient training with histogram-based tree method

10.2.5 Model Artifacts

The implementation produces two essential files for production:

- `credit_score_xgb_model.joblib`: Serialized trained model
- `preprocessing_artifacts.json`: Feature engineering parameters

10.2.6 Source Code

[Github Repositories](#)

10.3 Customer Churn Prediction System

10.3.1 Data Preparation

Dataset Loading

The study employs the Kaggle “Churn Modelling” dataset for reference. The datasets contain customer demographic and financial attributes relevant to churn behavior.

Listing 10.15: Load Datasets

```
1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4 import numpy as np
5
6 import xgboost as xgb
7 from sklearn.model_selection import train_test_split,
   GridSearchCV
8 from sklearn.model_selection import train_test_split
9 from sklearn.inspection import permutation_importance
10 from sklearn.metrics import confusion_matrix
11 from sklearn.metrics import ConfusionMatrixDisplay
12 import shap
13
14 train_data = pd.read_csv('/kaggle/input/playground-series-
   s4e1/train.csv')
15 test_data = pd.read_csv('/kaggle/input/playground-series-s4e1
   /test.csv')
16 sample_submission = pd.read_csv('/kaggle/input/playground-
   series-s4e1/sample_submission.csv')
17 original_data = pd.read_csv('/kaggle/input/bank-customer-
   churn-prediction/Churn_Modelling.csv')
```

Missing Value and Target Distribution Analysis

The datasets were assessed for completeness and class imbalance using visual inspection techniques. The target variable (*Exited*) exhibits imbalance typical of churn datasets.

Listing 10.16: Target Distribution Visualization

```
1 sns.countplot(x='Exited', data=train_data)
```



```
2 plt.show()
```

10.3.2 Feature Engineering

Categorical attributes were one-hot encoded, while non-informative identifiers (e.g., *id*, *CustomerId*, *Surname*) were removed. The data were subsequently partitioned into training and validation sets using stratified sampling to preserve class ratios.

Listing 10.17: Encoding and Data Split

```
1 X = train_data.drop(['id', 'CustomerId', 'Surname', 'Exited'],  
    axis=1)  
2 y = train_data['Exited']  
3  
4 X = pd.get_dummies(X)  
5 X_train, X_test, y_train, y_test = train_test_split(  
6     X, y, random_state=42, stratify=y)
```

10.3.3 Model Development

XGBoost, a gradient-boosted decision tree algorithm, was selected for its effectiveness in structured tabular data and imbalanced classification. Two models were trained: a baseline classifier and a tuned version incorporating regularization and class-weight adjustments.

Listing 10.18: Tuned XGBoost Classifier

```
1 clf_xgb = xgb.XGBClassifier(  
2     objective='binary:logistic',  
3     learning_rate=0.1,  
4     max_depth=5,  
5     gamma=1.0,  
6     reg_lambda=10.0,  
7     scale_pos_weight=5,  
8     subsample=0.9,  
9     colsample_bytree=0.5,  
10    seed=42)  
11  
12 clf_xgb.fit(
```

```
13 X_train, y_train,  
14 early_stopping_rounds=10,  
15 eval_metric='auc',  
16 eval_set=[(X_test, y_test)])
```

10.3.4 Model Evaluation

Performance was assessed using the confusion matrix and AUC-based early stopping. The model demonstrated strong discriminatory capability.

Listing 10.19: Confusion Matrix

```
1 pred = clf_xgb.predict(X_test)  
2 cm = confusion_matrix(y_test, pred)  
3 ConfusionMatrixDisplay(cm).plot()  
4 plt.show()
```

10.3.5 Prediction and Submission Generation

The trained model produced churn probabilities for the test set, which were submitted in accordance with Kaggle competition guidelines.

Listing 10.20: Generate Submission File

```
1 test_data = test_data.drop(['id', 'CustomerId', 'Surname'],  
    axis=1)  
2 test_data = pd.get_dummies(test_data)  
3  
4 sample_submission['Exited'] = clf_xgb.predict_proba(test_data  
   )[: , 1]  
5 sample_submission.to_csv('/kaggle/working/submission.csv',  
    index=False)
```



References

- [1] R. Anderson. *The Credit Scoring Toolkit*. Oxford University Press, 2007.
- [2] Janio Bachmann. Bank marketing dataset. <https://www.kaggle.com/datasets/janiobachmann/bank-marketing-dataset>, 2020. Kaggle dataset, accessed 2025-10-04.
- [3] Basel Committee on Banking Supervision. Basel iii: International framework for liquidity risk measurement, standards and monitoring. Technical report, Bank for International Settlements, 2010.
- [4] Basel Committee on Banking Supervision. Sound management of risks related to money laundering and financing of terrorism. Technical report, Bank for International Settlements, 2020.
- [5] Bitext. Bitext retail banking llm chatbot training dataset. <https://huggingface.co/datasets/bitext/Bitext-retail-banking-llm-chatbot-training-dataset>, 2023. Hugging Face dataset, accessed 2025-10-04.
- [6] H. P. Bunaes. 5 data science challenges banks face (and how to overcome them). <https://www.datarobot.com/blog/5-data-science-challenges-banks-face-and-how-to-overcome-them/>, April 2025. DataRobot Blog.
- [7] Federal Deposit Insurance Corporation. *Credit Risk Rating Systems*. FDIC Risk Management Manual of Examination Policies, 2024.
- [8] Deloitte. Banking & capital markets data and analytics market survey. Technical report, Deloitte Insights, 2024.
- [9] Shantanu Dhakadd. Bank customer churn prediction. <https://www.kaggle.com/datasets/shantanudhakadd/bank-customer-churn-prediction>, 2020. Kaggle dataset, accessed 2025-10-04.
- [10] FastAPI. Fastapi framework, high performance, easy to learn, fast to code, ready for production. <https://fastapi.tiangolo.com>, 2025. Accessed 2025.
- [11] Financial Action Task Force. International standards on combating money laundering and the financing of terrorism & proliferation. Technical report, FATF, Paris, 2023.

- [12] FirstEigen. Data integrity issues in banking: Compliance challenges & ai-driven solutions. <https://firsteigen.com/blog/compliance-challenges-due-to-data-integrity-issues/>, 2024. FirstEigen Blog.
- [13] Fortune Business Insights. Big data analytics market size, value & share analysis and forecast, 2023-2032. <https://www.fortunebusinessinsights.com/big-data-analytics-market-106179>, 2023. Accessed 2025.
- [14] Global Banking and Finance. Using behavioral analytics to strengthen customer retention in banking. <https://www.globalbankingandfinance.com/using-behavioral-analytics-to-strengthen-customer-retention-in-banking>, 2024. Global Banking and Finance.
- [15] IBM. Synthetic financial datasets for fraud detection, 2021. IBM Developer Resources.
- [16] IBM. Ai fraud detection in banking. <https://www.ibm.com/think/topics/ai-fraud-detection-in-banking>, January 2025. IBM Think.
- [17] Matellio Inc. Behavioral analytics in banking — the game-changer in the financial sector. <https://www.matellio.com/blog/behavioral-analytics-in-banking/>, 2024. Matellio Blog.
- [18] Informatica. Overcoming data integration challenges. <https://www.informatica.com/blogs/data-integration-challenges-and-problems.html>, 2024. Informatica Blog.
- [19] KPMG. Global anti-money laundering survey: Balancing effectiveness and efficiency. Technical report, KPMG International, 2021.
- [20] McKinsey & Company. Global banking annual review 2024: Attaining escape velocity. Technical report, McKinsey & Company, 2024. Accessed 2025.
- [21] Brajesh Mohapatra. Term deposit prediction data set. <https://www.kaggle.com/datasets/brajeshmohapatra/term-deposit-prediction-data-set/data>, 2021. Kaggle dataset, accessed 2025-10-04.
- [22] Financial Crimes Enforcement Network. *Suspicious Activity Report Filing Requirements*. U.S. Department of the Treasury, 2022.

- [23] Nexla. Overcome integration and data challenges of open banking. <https://nexla.com/open-banking-api/>, 2024. Nexla Blog.
- [24] Eastern Peak. Big data in the banking industry: The main challenges and use cases. <https://easternpeak.com/blog/big-data-in-banking-and-financial-services/>, 2023. Article updated on August 23, 2023.
- [25] PostgreSQL Global Development Group. About postgresql. <https://www.postgresql.org/about>, 2025. Accessed 2025.
- [26] J.P. Pressley. Why more banks are prioritizing integrated data management. <https://biztechmagazine.com/article/2024/10/why-more-banks-are-prioritizing-integrated-data-management>, October 2024. BizTech Magazine.
- [27] ResearchGate. Improve profiling bank customer's behavior using machine learning. https://www.researchgate.net/publication/335131761_Improve_Profiling_Bank_Customer's_Behavior_Using_Machine_Learning, 2019. ResearchGate.
- [28] ResearchGate. Ai-powered personalization in digital banking: A review of customer behavior analytics and engagement. https://www.researchgate.net/publication/391810532_AI-Powered_Personalization_In_Digital_Banking_A_Review_Of_Customer_Behavior_Analytics_And_Engagement, 2024. ResearchGate.
- [29] Rivery. Common data management challenges and solutions. <https://rivery.io/data-learning-center/data-management-challenges/>, April 2025. Rivery Data Learning Center.
- [30] RTInsights. 4 challenges for open banking integration. <https://www.rtinsights.com/4-challenges-for-open-banking-integration/>, 2024. RTInsights.
- [31] Hitachi Solutions. Modern data analytics in banking: Benefits, outlook & more. <https://global.hitachi-solutions.com/blog/big-data-banking/>, 2024. Hitachi Solutions Global.



- [32] TechMagic. Why use react for web development: 10 reasons to apply. <https://www.techmagic.co/blog/why-we-use-react-js-in-the-development>, 2025. Accessed 2025.
- [33] L. Thomas, D. Edelman, and J. Crook. *Credit Scoring and Its Applications*. SIAM, 2nd edition, 2017.
- [34] UCI Machine Learning Repository. Statlog (german credit data) data set, 2019. University of California, Irvine.
- [35] MLG ULB. Credit card fraud detection. <https://www.kaggle.com/datasets/mlg-ulb/creditcardfraud>, 2018. Kaggle dataset, accessed 2025-10-04.
- [36] Wolfsberg Group. Wolfsberg statement on developing an effective aml/ctf programme, 2020. The Wolfsberg Group.